

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN BẢO MẬT WEB VÀ ỨNG DỤNG

-&-

ĐỀ TÀI: PHÁT TRIỂN VÀ BẢO MẬT
ỨNG DỤNG ANDROID KOTLIN

Giảng viên hướng dẫn: **Ngô Khánh Khoa**

Thực hiện bởi Nhóm 2, gồm:

- | | | |
|--------------------------------|-----------------|--------------------|
| • LẠI QUAN THIÊN | 22521385 | Trưởng nhóm |
| • MAI NGUYỄN NAM PHƯƠNG | 22521164 | Thành viên |
| • HỒ DIỆP HUY | 22520541 | Thành viên |
| • NGUYỄN PHÚC NHI | 22521041 | Thành viên |

Lớp: **NT213.P11.ANTT**

TP. Hồ Chí Minh, tháng 12 năm 2024

LỜI MỞ ĐẦU

Trong thời đại công nghệ số phát triển mạnh mẽ, các ứng dụng di động đã và đang trở thành một phần không thể thiếu trong cuộc sống hàng ngày. Từ giao tiếp, giải trí đến các hoạt động thương mại, học tập và làm việc, ứng dụng di động đóng vai trò trung tâm trong việc kết nối con người với thế giới số. Theo thống kê, số lượng người dùng smartphone toàn cầu đang tăng trưởng không ngừng, kéo theo đó là sự bùng nổ của các ứng dụng trên các nền tảng như **Android** và **iOS**. Trong đó, Android chiếm phần lớn thị phần nhờ tính linh hoạt, khả năng tùy chỉnh cao và sự đa dạng về thiết bị.

Tuy nhiên, cùng với sự phát triển vượt bậc của ứng dụng di động, các nguy cơ về bảo mật ngày càng trở nên nghiêm trọng. Theo báo cáo của OWASP Mobile Top 10, các ứng dụng di động thường xuyên đối mặt với các lỗ hổng bảo mật như đánh cắp dữ liệu, giả mạo danh tính, mã độc hoặc truy cập trái phép vào các tài nguyên quan trọng. Đặc biệt, sự phân mảnh trong hệ sinh thái Android làm tăng thêm thách thức trong việc bảo đảm tính bảo mật cho từng thiết bị và phiên bản hệ điều hành khác nhau.

Nhận thấy tầm quan trọng của việc kết hợp bảo mật vào quá trình phát triển ứng dụng di động, nhóm chúng em đã lựa chọn thực hiện đề án **“Phát triển và Bảo mật ứng dụng Android Kotlin”**. Đề án này không chỉ nhằm phát triển một ứng dụng Android cơ bản bằng ngôn ngữ Kotlin, mà còn tập trung nghiên cứu và áp dụng các phương pháp bảo mật hiện đại để giảm thiểu các nguy cơ tiềm ẩn, hướng tới việc bảo vệ thông tin người dùng và nâng cao chất lượng ứng dụng.

Với mong muốn đóng góp một phần nhỏ vào việc phát triển ứng dụng di động an toàn hơn, nhóm hy vọng rằng kết quả của đề án sẽ mang lại những giá trị thiết thực và góp phần vào xu hướng bảo mật toàn diện trong thời đại công nghệ số.

LỜI CẢM ƠN

Trước tiên, chúng em xin gửi lời cảm ơn sâu sắc đến thầy Ngô Khánh Khoa - giảng viên hướng dẫn môn Bảo Mật Web và Ứng Dụng, đã tận tình chỉ dẫn và chia sẻ những kinh nghiệm quý báu trong suốt quá trình thực hiện đề tài. Những ý kiến đóng góp và sự hướng dẫn của thầy đã giúp chúng em rất nhiều trong việc định hướng và hoàn thiện nội dung đồ án.

Chúng em cũng xin gửi lời cảm ơn chân thành đến các thành viên trong nhóm, những người đã không ngừng nỗ lực, phối hợp và đóng góp ý kiến để hoàn thành đề tài một cách tốt nhất. Dù gặp nhiều khó khăn và giới hạn về thời gian, nhưng nhờ sự đoàn kết và tinh thần trách nhiệm, chúng em đã vượt qua và đạt được mục tiêu đề ra.

Cuối cùng, chúng em xin bày tỏ lòng biết ơn đến những người đã hỗ trợ và động viên chúng em trong suốt quá trình thực hiện đề tài. Do giới hạn về kiến thức và thời gian, không thể tránh khỏi những thiếu sót trong đồ án, chúng em rất mong nhận được sự góp ý từ thầy và các bạn để hoàn thiện hơn.

TP. Hồ Chí Minh, tháng 12 năm 2024

Nhóm 2, lớp NT213.P11.ANTT

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	1
1.1. Xu hướng phát triển ứng dụng di động.....	1
1.2. Phạm vi nghiên cứu và yêu cầu của đề tài	1
1.3. Tổng quan về ứng dụng Goatlin và mục tiêu dự án.....	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	3
2.1. Nền tảng phát triển ứng dụng Android	3
2.1.1. Tổng quan về hệ điều hành Android.....	3
2.1.2. Giới thiệu ngôn ngữ lập trình Kotlin	4
2.1.3. Kiến trúc ARM trên thiết bị di động.....	6
2.1.4. Android Studio.....	6
2.1.5. Hệ thống bảo mật sẵn có của Android.....	7
2.2. Thế nào là một ứng dụng di động an toàn.....	8
2.3. Top 10 OWASP Mobile Risks (2016).....	9
CHƯƠNG 3. PHÁT TRIỂN VÀ BẢO MẬT ỨNG DỤNG GOATLIN.....	11
3.1. Phát triển giao diện và trải nghiệm người dùng.....	11
3.2. Bảo mật ứng dụng Goatlin	13
3.2.1. Bảo mật Shared Preferences	13
3.2.2. Bảo mật RootDetectionHelper	18
3.2.3. M1: Improper Platform Usage (Sử dụng sai nền tảng)	22
3.2.4. M2: Insecure Data Storage (Lưu trữ dữ liệu không an toàn)	25
3.2.5. M3: Insecure Communication (Giao tiếp không an toàn).....	28
3.2.6. M4: Insecure Authentication (Xác thực không an toàn)	32
3.2.7. M5: Insufficient Cryptography (Mật mã học không đủ mạnh)	37
3.2.8. M6: Insecure Authorization (Ủy quyền không an toàn).....	42

3.2.9. M7: Client Code Quality (Chất lượng mã phía client kém)	44
3.2.10. M8: Code Tampering (Sử đổi mã).....	47
3.2.11. M9: Reverse Engineering (Kỹ thuật đảo ngược).....	50
3.2.12. M10: Extraneous Functionality (Tính năng thừa hoặc không cần thiết).....	54
CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ.....	56
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	61
5.1. Kết luận	61
5.2. Hạn chế.....	61
5.3. Hướng phát triển trong tương lai	62
5.4. Lời kết.....	63

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Xu hướng phát triển ứng dụng di động

Lĩnh vực phát triển ứng dụng di động hiện đang phát triển nhanh chóng với nhiều xu hướng mới. Một trong những xu hướng đó là việc sử dụng mạng 5G với mục đích nâng cao trải nghiệm truy cập Internet nhờ vào tốc độ nhanh hơn, độ trễ thấp và giúp các nhà phát triển tạo ra các ứng dụng tương tác và phản hồi nhanh hơn.

Bên cạnh đó, trí tuệ nhân tạo (AI) và học máy (ML) được tích hợp nhiều hơn trong phát triển ứng dụng di động để góp phần phát triển các ứng dụng thông minh và có tính cá nhân hóa. Internet of Things (IoT) cũng là một trong những xu hướng mới với nhiều ứng dụng tích hợp các thiết bị thông minh như các quản lý nhà cửa hay theo dõi sức khỏe,...

Hiện nay nhu cầu về các nền tảng phát triển low-code hoặc no-code cũng đang tăng cao, nó giúp những người không chuyên về mặt kỹ thuật có thể dễ dàng tạo và phát triển ứng dụng mà không cần phụ thuộc vào các nhà phát triển chuyên nghiệp.

1.2. Phạm vi nghiên cứu và yêu cầu của đề tài

Phạm vi nghiên cứu của đề tài này chủ yếu tập trung vào phân tích bảo mật cho ứng dụng Android được viết bằng ngôn ngữ lập trình Kotlin. Trong nghiên cứu này, ta sẽ kiểm tra các lỗ hổng bảo mật phổ biến như SQL Injection, Cross-Site Scripting (XSS), lưu trữ không an toàn (insecure storage) và các quyền truy cập không cần thiết,... Các công cụ bảo mật và môi trường phát triển như Android Studio, Docker cùng các công nghệ liên quan khác sẽ được sử dụng để triển khai ứng dụng và thiết lập môi trường kiểm thử.

Sau khi khắc phục các lỗ hổng bảo mật đã được phát hiện, ứng dụng sẽ được xây dựng lại và kiểm thử để đảm bảo ứng dụng an toàn và tuân thủ nghiêm ngặt các nguyên tắc bảo mật. Ngoài ra, đề tài cũng tập trung vào việc nâng cấp giao diện người dùng (UI/UX) và phát triển thêm các tính năng mới cho ứng dụng nhằm giúp người dùng có trải nghiệm tốt hơn.

Yêu cầu của đề tài bao gồm việc tìm kiếm, phân tích và đánh giá các lỗ hổng bảo mật tiềm ẩn trong ứng dụng Goatlin. Kết quả cuối cùng là tạo ra một ứng dụng ứng

dụng hoàn thiện, không chỉ thân thiện với người dùng mà còn đáp ứng được các tiêu chuẩn bảo mật.

1.3. Tổng quan về ứng dụng Goatlin và mục tiêu dự án

Ứng dụng Goatlin được thiết kế để tạo ra một môi trường thử nghiệm với các lỗ hổng bảo mật cố ý để người dùng có thể thực hành phát hiện và khắc phục các lỗ hổng này, từ đó hiểu rõ hơn về các rủi ro tiềm ẩn trong ứng dụng di động và nâng cao kỹ năng bảo mật.

Mục tiêu chính của dự án Goatlin là phát hiện, vá các lỗ hổng bảo mật và hoàn thiện ứng dụng Android Kotlin. Goatlin cung cấp một nền tảng học tập thực tế, giúp người dùng nắm vững các kỹ thuật bảo mật và áp dụng chúng vào việc phát triển ứng dụng di động một cách an toàn. Thông qua việc thực hành với các lỗ hổng bảo mật trong dự án này, người dùng sẽ sẵn sàng cho các tình huống thực tế do đã trải nghiệm giải quyết các vấn đề bảo mật trong môi trường được kiểm soát.

Liên kết dẫn đến mã nguồn của dự án Goatlin gốc:

<https://github.com/Checkmarx/Goatlin.git>

Liên kết dẫn đến mã nguồn của dự án Goatlin đã được nhóm phát triển:

<https://github.com/WanThinnn/Goatlin.git>

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Nền tảng phát triển ứng dụng Android

2.1.1. Tổng quan về hệ điều hành Android

Android là một hệ điều hành dựa trên nền tảng Linux được thiết kế dành cho các thiết bị di động có màn hình cảm ứng như điện thoại thông minh và máy tính bảng. Android sở hữu mã nguồn mở nên lập trình viên có thể dễ dàng điều chỉnh và phân phối nó một cách tự do. Ban đầu, Android được phát triển bởi Android Inc. với sự hỗ trợ tài chính từ Google và sau này được chính Google mua lại vào năm 2005.

Kiến trúc của hệ điều hành Android:

- Tầng Applications: Là tầng chứa các ứng dụng Danh bạ, Gọi điện, Trình duyệt, Nghe nhạc,... các ứng dụng này thường có sẵn trên thiết bị mới.

- Tầng Framework: Là tầng chứa các API để làm việc với hệ điều hành như lấy thông tin danh bạ, quản lý các Activity, quản lý địa điểm, quản lý các View.

- Tầng Libraries: Chứa các thư viện, API gần như là cốt lõi của Android, bao gồm bộ quản lý bề mặt cảm ứng (Surface Manager), OpenGL (phục vụ cho việc dựng đồ họa phức tạp),...

- Tầng Android Runtime: Chứa các thư viện lõi của Android và máy ảo Dalvik Virtual Machine (từ Android 4 trở lên có thêm máy ảo ART).

- Tầng Kernel: Là nhân lõi của hệ điều hành, chứa các tập lệnh, driver giao tiếp giữa phần cứng và phần mềm của Android.



Các phiên bản của Android: Android bắt đầu với phiên bản 1.0 (2008), mang đến những tính năng cơ bản như trình duyệt web, tích hợp Gmail, YouTube và Google Maps, đánh dấu sự ra đời của một hệ điều hành di động mở. Sau đó các phiên bản ban đầu như Cupcake (1.5) và Donut (1.6) lần lượt ra đời với bàn phím ảo, widget và hỗ trợ nhiều độ phân giải màn hình, đặt nền móng cho sự phát triển của Android. Upside Down Cake (14) ra đời năm 2023, tập trung vào bảo mật, tối ưu hiệu suất, hỗ trợ tốt hơn cho các thiết bị hiện đại như màn hình gập và màn hình lớn đồng thời mang lại trải nghiệm cá nhân hóa hơn. Tính đến 2024, phiên bản mới nhất là Vanilla Ice Cream (15). Android 15 là bước nhảy vọt về tính năng và hiệu năng so với các phiên bản tiền nhiệm.

2.1.2. Giới thiệu ngôn ngữ lập trình Kotlin

Kotlin là ngôn ngữ lập trình tĩnh (static) do JetBrains phát triển. Ngôn ngữ này đã tạo nên các IDE như IntelliJ IDEA, PhpStorm, Appcode,... Kotlin được JetBrains giới thiệu lần đầu tiên vào năm 2011 và là ngôn ngữ mới cho JVM. Kotlin là ngôn ngữ hướng đối tượng (object-oriented) và là “ngôn ngữ tốt hơn” so với Java nhưng vẫn có khả năng tương tác hoàn toàn với code Java. Kotlin được tài trợ bởi Google, được công bố là một trong những ngôn ngữ chính thức cho Android Development năm 2017.

Ưu điểm của ngôn ngữ lập trình Kotlin:

- Kotlin khắc phục các điểm yếu của Java: Java là ngôn ngữ lập trình được ưu tiên sử dụng với lập trình viên thực hiện các dự án Android. Nhưng Java vẫn có những hạn chế như thiếu khả năng mở rộng, không hỗ trợ tính năng cho các lập trình hàm. Bên cạnh đó, Java mới chỉ có thể hỗ trợ một phần các tính năng Android tại Java 8. Chính vì vậy, Kotlin ra đời được đánh giá giúp khắc phục hoàn toàn các hạn chế mà Java không thực hiện được. Ngoài ra, Kotlin thừa hưởng các đặc tính từ Java nên giúp người dùng khai thác mọi nền tảng từ Java class Library.

- Mã nguồn được tối ưu hóa, dễ hiểu và dễ triển khai: Ngôn ngữ lập trình Kotlin được xây dựng với hệ thống code ngắn gọn giúp lập trình viên dễ triển khai. Kotlin là ngôn ngữ lập trình có mục đích nâng cao hơn so với Java, chứ không phải hoàn toàn viết lại. Do đó, kinh nghiệm lập trình Java có thể được áp dụng trong dự án Kotlin.

- Không gặp lỗi NullPointerException: Trong Java, lỗi NullPointerException thường xuyên xuất hiện ngay khi gán giá trị null đến một đối tượng, truy xuất và phát hiện lỗi. Trong khi đó, Kotlin được thiết kế để loại bỏ hầu hết các nguồn tham chiếu null dựa trên cơ chế Null-safety. Do đó, Kotlin an toàn hơn nhiều so với Java.

- Kotlin sở hữu khả năng tương tác cao: Kotlin tương thích hoàn toàn với Java trên một dự án, có thể sử dụng cho cả Kotlin và Java.

- Kotlin được hỗ trợ ưu tiên trong Android Studio và IDE: Hiện nay, các lập trình viên Android đều có thể tận dụng IDE tích hợp từ Android Studio 3.0. Các IDE hỗ trợ Java đều hỗ trợ Kotlin. Kotlin được hỗ trợ ưu tiên trong Android Studio và IDE giúp lập trình viên dễ dàng thực hiện công việc hơn.

Nhược điểm của ngôn ngữ lập trình Kotlin:

- Chưa có kiểu Aliases: Vì Kotlin chưa có kiểu Aliases nên phải viết kiểu hàm thủ công, vì vậy phần mã nguồn sẽ bị dư thừa.

- Các mặc định class trong Kotlin là final: Cần phải thêm từ khóa Open nếu như muốn class final trở thành class thông thường như trong Java. Đây là hạn chế có thể có trong các dự án có mã nguồn kết hợp giữa Kotlin và Java. Bởi vì một số Java Framework thường tự động bỏ qua từ khóa Final trong mã của Kotlin. Điều này sẽ khiến cho Kotlin có thể chạy không đúng với mục đích ban đầu.

- Cộng đồng hỗ trợ hạn chế: Vì Kotlin là một ngôn ngữ tương đối mới, nên cộng đồng Kotlin vẫn còn khá nhỏ, đặc biệt là so với cộng đồng của ngôn ngữ khác như Java.
- Kotlin không tự ép kiểu dữ liệu: Kotlin sẽ không thực hiện tự động ép kiểu với những dữ liệu thuộc dạng nguyên thủy.

2.1.3. Kiến trúc ARM trên thiết bị di động

ARM là viết tắt của cụm từ Advanced RISC Machine hay Acorn RISC Machine. Đây là một kiến trúc bộ xử lý dựa trên RISC (Reduced Instruction Set Computer) do Arm Holdings, Ltd. phát triển.

Kiến trúc RISC thường tiêu thụ ít tài nguyên bán dẫn hơn so với kiến trúc CISC. Do đó, chip ARM có thể loại bỏ các lệnh không cần thiết, thực hiện nhiều lệnh trong mỗi chu kỳ xung nhằm giảm tiêu thụ điện năng, diện tích và cung cấp hiệu suất cao cho các thiết bị di động như điện thoại, laptop,...

R trong ARM là đại diện của RISC, một máy tính có tập lệnh đơn giản hóa, giúp thực hiện mọi chức năng một cách hiệu quả trên một con chip duy nhất. Tập lệnh đơn giản cũng đồng nghĩa với việc bộ xử lý có thể mã hóa chúng bằng số lượng bit ít hơn, giảm tiêu hao bộ nhớ và thời gian chu kỳ thực thi.

2.1.4. Android Studio

Android Studio là một môi trường tích hợp phát triển (Integrated Development Environment – IDE) được phát triển bởi Google dành cho việc phát triển ứng dụng trên nền tảng Android. Nó cung cấp một loạt các công cụ và tính năng để giúp nhà phát triển xây dựng và triển khai ứng dụng Android một cách dễ dàng.

Công cụ này cung cấp một giao diện trực quan và các trình biên dịch, trình gỡ lỗi, trình tạo giao diện người dùng, trình quản lý phiên bản và nhiều công cụ khác. Nó hỗ trợ viết code trong ngôn ngữ Java hoặc Kotlin và tích hợp sẵn các thư viện và công cụ phát triển Android. Một trong những tính năng quan trọng của công cụ này là khả năng sử dụng Gradle, một hệ thống quản lý dự án mạnh mẽ. Gradle cho phép quản lý phụ thuộc, xây dựng, kiểm thử và đóng gói ứng dụng Android một cách linh hoạt và hiệu quả.

Android Studio đã được công bố lần đầu tiên vào năm 2013 tại hội nghị Google I/O và chính thức phát hành rộng rãi vào năm 2014. Trước đó, các nhà phát triển ứng dụng Android thường sử dụng các công cụ như Eclipse IDE hoặc IDE của Java để phát triển ứng dụng. Sau khi được công bố công cụ này đã trở thành một công cụ phát triển ứng dụng Android chính thức và được Google khuyến nghị sử dụng. Nó đã thay thế Eclipse IDE và trở thành môi trường phát triển chính cho các dự án Android.

2.1.5. Hệ thống bảo mật sẵn có của Android

Android kết hợp các tính năng bảo mật hàng đầu trong ngành và hợp tác với các nhà phát triển cũng như người triển khai thiết bị để giữ cho nền tảng và hệ sinh thái Android được an toàn. Một mô hình bảo mật mạnh mẽ là điều cần thiết để tạo ra một hệ sinh thái ứng dụng và thiết bị mạnh mẽ được xây dựng trên và xung quanh nền tảng Android và được hỗ trợ bởi các dịch vụ đám mây. Kết quả là, trong toàn bộ vòng đời phát triển của mình, Android đã phải tuân theo một chương trình bảo mật nghiêm ngặt.

Android được thiết kế để mở. Các ứng dụng Android sử dụng phần cứng và phần mềm tiên tiến cũng như dữ liệu cục bộ và dữ liệu được cung cấp, được cung cấp thông qua nền tảng để mang lại sự đổi mới và giá trị cho người tiêu dùng. Để hiện thực hóa giá trị đó, nền tảng cung cấp môi trường ứng dụng bảo vệ tính bảo mật, tính toàn vẹn và tính khả dụng của người dùng, dữ liệu, ứng dụng, thiết bị và mạng.

Android mong muốn trở thành hệ điều hành an toàn và hữu dụng nhất dành cho nền tảng di động bằng cách tái sử dụng các biện pháp kiểm soát bảo mật hệ điều hành truyền thống để:

- Bảo vệ ứng dụng và dữ liệu người dùng
- Bảo vệ tài nguyên hệ thống (bao gồm cả mạng)
- Cung cấp cách ly ứng dụng khỏi hệ thống, các ứng dụng khác và khỏi người dùng

Để đạt được những mục tiêu này, Android có các tính năng bảo mật tích hợp giúp giảm đáng kể tần suất và tác động của các vấn đề về bảo mật ứng dụng. Hệ thống này được thiết kế để có thể tạo ứng dụng có các quyền mặc định đối với hệ thống và tệp, đồng thời tránh được những quyết định khó khăn về bảo mật:

- Bảo mật mạnh mẽ ở cấp độ hệ điều hành thông qua nhân Linux
- Application Sandbox bắt buộc cho tất cả ứng dụng
- Giao tiếp liên tiến trình an toàn (Secure IPC)
- Ký ứng dụng
- Quyền do ứng dụng xác định và do người dùng cấp

2.2. Thế nào là một ứng dụng di động an toàn

Ứng dụng di động ngày càng trở nên phổ biến, nhưng đi kèm với đó là nhiều thách thức về bảo mật:

- **Quyền truy cập dữ liệu nhạy cảm:** Ứng dụng di động thường yêu cầu quyền truy cập vào dữ liệu nhạy cảm như danh bạ, tin nhắn và vị trí. Điều này có thể dẫn đến rủi ro rò rỉ thông tin cá nhân nếu không được bảo vệ đúng cách.

- **Ứng dụng giả mạo và độc hại:** Người dùng có thể bị lừa tải xuống các ứng dụng giả mạo hoặc độc hại, dẫn đến việc cung cấp quyền truy cập vào dữ liệu nhạy cảm.

- **Mã hóa dữ liệu:** Đảm bảo dữ liệu được mã hóa cả khi lưu trữ và truyền tải là rất quan trọng để duy trì tính bảo mật và toàn vẹn của dữ liệu.

- **Xác thực và ủy quyền:** Triển khai các cơ chế xác thực mạnh mẽ và kiểm soát ủy quyền để đảm bảo chỉ những người dùng được phép mới có thể truy cập vào các chức năng và dữ liệu quan trọng.

- **Cập nhật và vá lỗi bảo mật:** Luôn cập nhật các bản vá bảo mật và tuân thủ các tiêu chuẩn và quy định để bảo vệ ứng dụng khỏi các lỗ hổng mới.

OWASP Mobile Top 10 là danh sách các lỗ hổng bảo mật phổ biến nhất trong ứng dụng di động. Việc tích hợp các hướng dẫn từ OWASP Mobile Top 10 trong quy trình phát triển ứng dụng giúp các nhà phát triển nhận diện và giải quyết các lỗ hổng cũng như các thách thức về bảo mật:

- **Nhận diện các lỗ hổng phổ biến:** OWASP Mobile Top 10 giúp xác định các lỗ hổng bảo mật phổ biến nhất trong ứng dụng di động, từ đó giúp các nhà phát triển tập trung vào việc khắc phục những điểm yếu này.

- **Tăng cường bảo mật:** Bằng cách tuân thủ các hướng dẫn của OWASP Mobile Top 10, nhà phát triển có thể tăng cường bảo mật cho ứng dụng di động của mình, bảo vệ dữ liệu người dùng và duy trì tuân thủ các quy định.

- **Xây dựng niềm tin với người dùng:** Việc đảm bảo ứng dụng di động an toàn không chỉ bảo vệ dữ liệu mà còn xây dựng niềm tin của người dùng đối với ứng dụng.

Một ứng dụng di động an toàn là ứng dụng được thiết kế, phát triển và triển khai với các biện pháp bảo vệ mạnh mẽ để bảo vệ dữ liệu người dùng, ngăn chặn các cuộc tấn công mạng và đảm bảo tính toàn vẹn của hệ thống. Ứng dụng này phải đáp ứng các yêu cầu bảo mật cần thiết để giảm thiểu rủi ro từ các mối đe dọa ngày càng phức tạp trong môi trường di động. Việc tuân thủ các hướng dẫn từ OWASP Mobile Top 10 giúp các nhà phát triển nhận diện và khắc phục các lỗ hổng bảo mật phổ biến, từ đó phát triển các ứng dụng di động an toàn.

2.3. Top 10 OWASP Mobile Risks (2016)

Top 10 Mobile Risks 2016 do OWASP công bố bao gồm các mối nguy cơ hàng đầu ảnh hưởng đến bảo mật của ứng dụng di động:

M1 - Improper Platform Usage: Lỗ hổng này xuất hiện khi các nhà phát triển không tuân thủ các tiêu chuẩn hoặc cách sử dụng đúng đắn của nền tảng di động, chẳng hạn như sử dụng sai API, không bảo vệ quyền truy cập, hoặc sử dụng các tính năng hệ điều hành không an toàn. Điều này có thể dẫn đến lỗ hổng bảo mật, như lạm dụng quyền hoặc lỗi cấu hình.

M2 - Insecure Data Storage: Ứng dụng di động có thể lưu trữ thông tin nhạy cảm (như mật khẩu, token, hoặc thông tin cá nhân) mà không mã hóa hoặc sử dụng cơ chế bảo mật yếu. Điều này khiến dữ liệu dễ bị truy cập trái phép khi thiết bị bị root, jailbreak, hoặc mất cắp.

M3 - Insecure Communication: Khi ứng dụng di động truyền dữ liệu qua mạng (như Wi-Fi hoặc di động), sử dụng các giao thức không bảo mật hoặc thiếu mã hóa có thể khiến dữ liệu nhạy cảm bị đánh cắp qua tấn công trung gian (MITM).

M4 - Insecure Authentication: Các cơ chế xác thực không đúng cách, chẳng hạn như lưu trữ thông tin xác thực cục bộ hoặc thiếu kiểm tra bảo mật, có thể khiến kẻ tấn công giả mạo danh tính người dùng hợp pháp.

M5 - Insufficient Cryptography: Việc sử dụng thuật toán mã hóa lỗi thời, cấu hình không đúng cách, hoặc tự thiết kế thuật toán có thể làm cho dữ liệu được mã hóa trở nên dễ dàng bị giải mã và khai thác.

M6 - Insecure Authorization: Khi ứng dụng không thực thi đúng cách các quyền hạn và kiểm tra truy cập, kẻ tấn công có thể tận dụng để thực hiện các hành động không được phép hoặc truy cập dữ liệu nhạy cảm.

M7 - Client Code Quality: Các lỗi lập trình như tràn bộ đệm, xử lý ngoại lệ không đúng cách, hoặc logic sai trong mã nguồn của ứng dụng có thể bị khai thác để thực hiện các cuộc tấn công.

M8 - Code Tampering: Kẻ tấn công có thể sửa đổi mã nguồn hoặc dữ liệu của ứng dụng (như thêm mã độc hoặc thay đổi hành vi ứng dụng) và phân phối phiên bản không chính thức để lừa đảo hoặc gây hại.

M9 - Reverse Engineering: Ứng dụng di động dễ bị đảo ngược để khám phá mã nguồn, logic ứng dụng, hoặc thông tin nhạy cảm như khóa API. Kẻ tấn công sử dụng công cụ như APKTool, JD-GUI để phân tích và khai thác.

M10 - Extraneous Functionality: Ứng dụng có thể chứa các tính năng ẩn hoặc chức năng không cần thiết trong môi trường sản xuất (như tài khoản thử nghiệm, giao diện gỡ lỗi) mà kẻ tấn công có thể tận dụng để xâm nhập hệ thống.

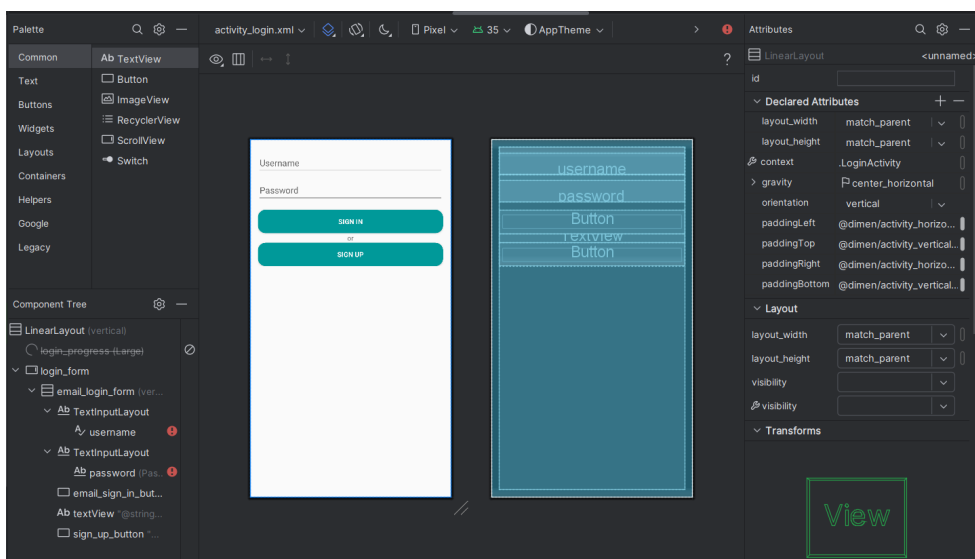
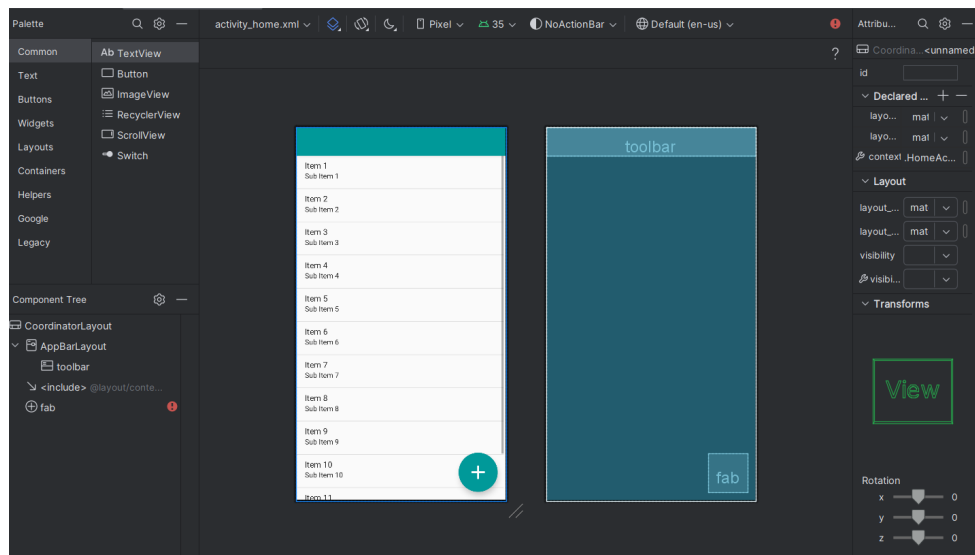
CHƯƠNG 3. PHÁT TRIỂN VÀ BẢO MẬT ỨNG DỤNG GOATLIN

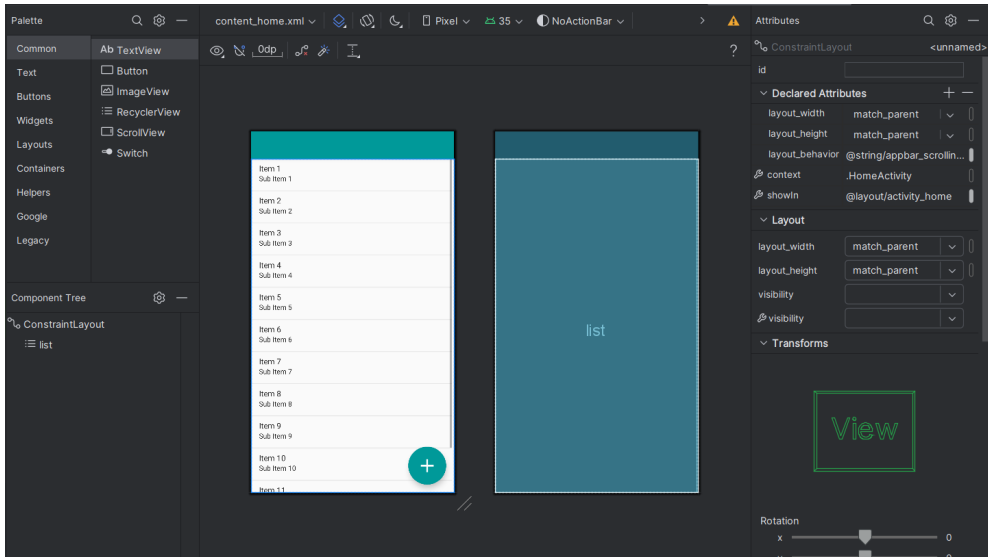
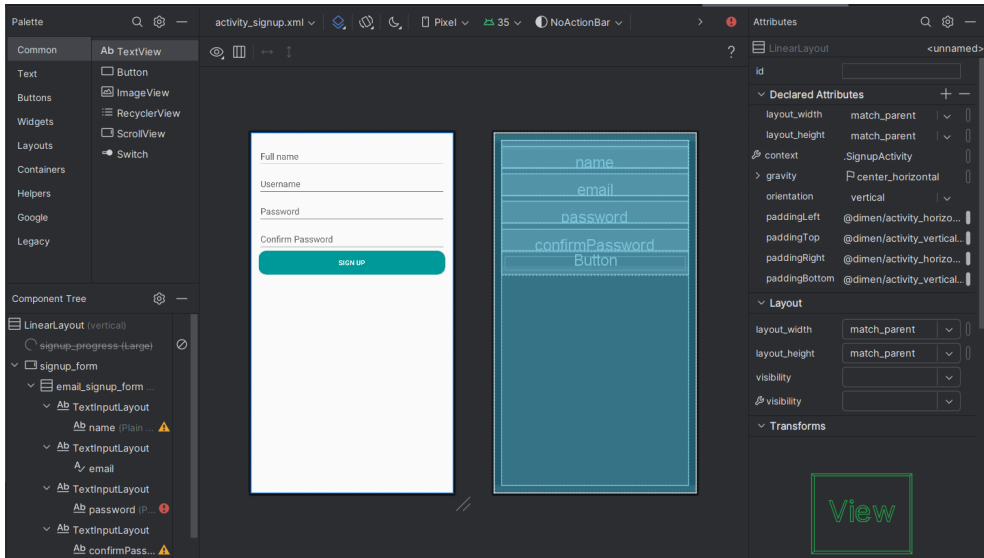
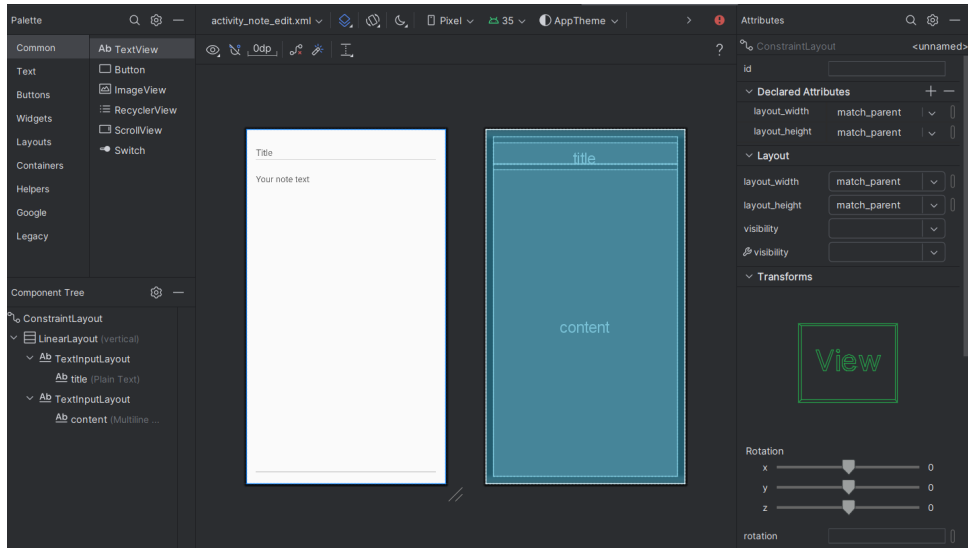
3.1. Phát triển giao diện và trải nghiệm người dùng

Giao diện của ứng dụng Goatlin đã được nhóm chỉnh sửa lại cho phù hợp với thị hiếu và xu hướng giao diện hiện nay hơn.

- Các button đều được bo góc nhẹ, nhằm làm “dịu mắt” người dùng
- Màu sắc chủ đạo của ứng dụng là xanh, không quá rực rỡ và thích hợp với mọi lứa tuổi sử dụng ứng dụng.
- Thanh toolbar được đồng nhất với thanh toolbar hệ điều hành, nhằm mang đến trải nghiệm liền lạc trong ứng dụng

Dưới đây mà một số hình ảnh thiết kế giao diện được chụp ra từ mã nguồn.





3.2. Bảo mật ứng dụng Goatlin

3.2.1. Bảo mật *Shared Preferences*

a. Phân tích:

Mô tả lỗ hổng: Lớp PreferenceHelper là một lớp hỗ trợ lưu trữ và truy xuất dữ liệu trong ứng dụng Android thông qua SharedPreferences, nhưng đoạn code ban đầu chỉ đưa lên gợi ý một đối tượng PreferenceHelper trong Kotlin để quản lý các thao tác với SharedPreferences, nếu thiết bị người dùng sử dụng quyền admin để truy cập vào bộ nhớ thiết bị thì họ có thể thấy được các thông tin SharedPreferences đã được lưu

Mức độ ảnh hưởng của lỗ hổng: rất nghiêm trọng khi các dữ liệu nhạy cảm có thể dễ dàng bị tìm thấy và hiểu được

b. Khắc phục:

Ta sẽ thực hiện thiết kế lại lớp PreferenceHelper này bằng cách sử dụng các cải tiến như sau

- **Mã hóa dữ liệu lưu trữ:** Sử dụng AES/GCM/NoPadding để mã hóa dữ liệu trước khi lưu

- **Sử dụng Keystore để bảo mật khóa mã hóa:** Khóa mã hóa (SecretKey) được quản lý bởi AndroidKeyStore, tránh lưu khóa trực tiếp trong mã nguồn. Khóa này được lưu trữ trong một phần cứng bảo mật chuyên dụng hoặc trong một vùng bảo mật của hệ điều hành tùy phiên bản Android

- **Cung cấp các hàm tiện ích:** Các hàm tiện ích để lưu dữ liệu, đọc dữ liệu, xóa dữ liệu theo key...

Cụ thể các hàm được ta cung cấp sẽ như sau

- Hàm **init(context: Context)**

```
fun init(context: Context) {  
    if (!this::sharedPreferences.isInitialized) {  
        sharedPreferences = context.getSharedPreferences("${context.packageName}_secure", Context.MODE_PRIVATE)  
  
        // Tạo khóa nếu chưa có  
        createKeyIfNecessary()  
    }  
}
```

- Khởi tạo SharedPreferences nếu chưa được khởi tạo.
- Tạo khóa trong Keystore nếu chưa tồn tại.
- Luồng hoạt động:
 - i. Kiểm tra xem sharedPreferences đã được khởi tạo hay chưa.
 - ii. Nếu chưa, tạo SharedPreferences với tên <tên package>_secure.
 - iii. Gọi createKeyIfNecessary để tạo khóa.

- Hàm **createKeyIfNecessary()**

```
// Tạo khóa trong Keystore nếu chưa tồn tại  
private fun createKeyIfNecessary() {  
    val keyStore = KeyStore.getInstance( type: "AndroidKeyStore")  
    keyStore.load( param: null)  
  
    if (!keyStore.containsAlias(KEY_ALIAS)) {  
        val keyGenerator = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, provider: "AndroidKeyStore")  
        val keyGenSpec = KeyGenParameterSpec.Builder(  
            KEY_ALIAS,  
            purposes: KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT  
        )  
            .setBlockModes(KeyProperties.BLOCK_MODE_GCM)  
            .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)  
            .build()  
  
        keyGenerator.init(keyGenSpec)  
        keyGenerator.generateKey()  
    }  
}
```

- Tạo khóa mã hóa trong Android Keystore nếu chưa có.
- Luồng hoạt động:
 - i. Tải KeyStore.

- ii. Kiểm tra xem khóa có tồn tại với alias KEY_ALIAS (định danh khóa)
- iii. Nếu chưa, sử dụng KeyGenerator để tạo khóa mới với cấu hình:
 - Thuật toán AES.
 - Chế độ GCM.
 - Không sử dụng padding

- Hàm **getSecretKey()**

```
// Lấy SecretKey từ Keystore
private fun getSecretKey(): SecretKey {
    val keyStore = KeyStore.getInstance( type: "AndroidKeyStore")
    keyStore.load( param: null)
    return if (keyStore.containsAlias(KEY_ALIAS)) {
        (keyStore.getEntry(KEY_ALIAS, protParam: null) as KeyStore.SecretKeyEntry).secretKey
    } else {
        createKeyIfNecessary()
        (keyStore.getEntry(KEY_ALIAS, protParam: null) as KeyStore.SecretKeyEntry).secretKey
    }
}
```

- Lấy khóa mã hóa từ Keystore.
- Nếu khóa chưa tồn tại, gọi createKeyIfNecessary để tạo khóa.
- Luồng hoạt động:
 - i. Tải KeyStore.
 - ii. Kiểm tra và lấy khóa từ Keystore.
 - iii. Nếu khóa không tồn tại, tạo khóa mới.

- Hàm **setString(key: String, value: String)**

```
// Lưu dữ liệu mã hóa vào SharedPreferences
fun setString(key: String, value: String) {
    val cipher = Cipher.getInstance(TRANSFORMATION)
    cipher.init(Cipher.ENCRYPT_MODE, getSecretKey())
    val iv = cipher.iv // Initialization Vector (IV)
    val encryptedBytes = cipher.doFinal(value.toByteArray())

    // Lưu IV và dữ liệu mã hóa dưới dạng chuỗi
    val ivBase64 = Base64.encodeToString(iv, Base64.DEFAULT)
    val encryptedBase64 = Base64.encodeToString(encryptedBytes, Base64.DEFAULT)
    val combinedValue = "$ivBase64:$encryptedBase64"

    sharedPreferences.edit().putString(key, combinedValue).apply()
}
```

- Mã hóa giá trị trước khi lưu vào SharedPreferences.
- Luồng hoạt động:
 - i. Khởi tạo Cipher ở chế độ mã hóa (ENCRYPT_MODE).
 - ii. Sinh IV (Initialization Vector) và mã hóa dữ liệu.
 - iii. Kết hợp IV và dữ liệu đã mã hóa thành một chuỗi.
 - iv. Lưu chuỗi này vào SharedPreferences.

- Hàm getString(key: String, default: String?)

```
// Đọc và giải mã dữ liệu từ SharedPreferences
fun getString(key: String, default: String? = null): String {
    val encryptedData = sharedPreferences.getString(key, null) ?: return default ?: ""
    val parts = encryptedData.split(...delimiters: ":")
    if (parts.size != 2) return default ?: ""

    val iv = Base64.decode(parts[0], Base64.DEFAULT)
    val encryptedBytes = Base64.decode(parts[1], Base64.DEFAULT)

    try {
        val cipher = Cipher.getInstance(TRANSFORMATION)
        val spec = GCMParameterSpec(tLen: 128, iv)
        cipher.init(Cipher.DECRYPT_MODE, getSecretKey(), spec)
        return String(cipher.doFinal(encryptedBytes))
    } catch (e: Exception) {
        e.printStackTrace() // Ghi log để kiểm tra lỗi, không để lộ thông tin nhạy cảm
        return default ?: ""
    }
}
```

- Giải mã và trả về giá trị từ SharedPreferences.

- Luồng hoạt động:
 - i. Lấy chuỗi mã hóa từ SharedPreferences.
 - ii. Phân tách chuỗi thành IV và dữ liệu đã mã hóa.
 - iii. Sử dụng IV và khóa mã hóa để giải mã.
 - iv. Trả về chuỗi giải mã, hoặc giá trị mặc định nếu có lỗi.

- Hàm **removeKey(key: String)**

```
// Xóa dữ liệu khỏi SharedPreferences
fun removeKey(key: String) {
    sharedPreferences.edit().remove(key).apply()
}
```

- Xóa một khóa cụ thể trong SharedPreferences

+ Hàm **clearAll()**

```
// Xóa tất cả dữ liệu trong SharedPreferences
fun clearAll() {
    sharedPreferences.edit().clear().apply()
}
```

- Xóa toàn bộ dữ liệu trong SharedPreferences

c. Tác dụng:

Sau khi thiết kế lại lớp Shared Preferences để lớp này cung cấp các chức năng liên quan đến việc lưu trữ, đọc và xóa dữ liệu mã hóa trong SharedPreferences thì đã đảm bảo được cho người dùng:

- **Bảo vệ dữ liệu người dùng:** Mã hóa giúp bảo vệ thông tin nhạy cảm như mật khẩu, token xác thực, và các dữ liệu cá nhân khác khỏi bị truy cập trái phép
- **Ngăn chặn tấn công:** Khi dữ liệu được mã hóa, kẻ tấn công sẽ gặp khó khăn hơn trong việc truy cập và sử dụng thông tin, ngay cả khi họ có thể truy cập vào thiết bị
- **Giảm thiểu rủi ro pháp lý:** Bảo vệ dữ liệu cá nhân giúp doanh nghiệp tránh được các vấn đề pháp lý và phạt tiền liên quan đến vi phạm bảo mật

d. Kết quả:

Khi thực hiện kiểm thử thì giá trị được lưu theo khóa bên trong đường dẫn data/data/com.vulnerablekotlinapp/shared_pref sẽ như sau:

```
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="userName">MvH9S1nqmQo3TPfP&#10;;Gau8jz2yks1R9xlKiP54JBjtPAEKZ6h5Ka8=&#10;;
  <string name="userId">XsLPyK60q/HDXyqZ&#10;;G42POv6CNKgA4TBoeC3E2xM=&#10;;
</map>
```

3.2.2. Bảo mật RootDetectionHelper

a. Phân tích:

Mô tả lỗ hổng: Ngoài các lớp được định nghĩa sẵn của chương trình thì nhóm nhận thấy ta cần thêm một lớp được dùng để ngăn các thiết bị đã root sử dụng tới ứng dụng, bởi vì các thiết bị này có thể dễ dàng bị tấn công và truy cập trái phép vào dữ liệu nhạy cảm, đồng thời nó cũng có thể cài đặt và chạy các ứng dụng không được kiểm duyệt, tăng nguy cơ nhiễm phần mềm độc hại và ngoài ra còn thay đổi các tệp hệ thống và cài đặt các bản vá không chính thức, có thể làm thay đổi hoặc phá vỡ chức năng của ứng dụng

Mức độ ảnh hưởng của lỗ hổng: rất nghiêm trọng vì thiết bị root có thể thực hiện nhiều phương pháp gây ảnh hưởng đến an toàn cũng như uy tín của ứng dụng

b. Khắc phục:

Ta sẽ thực hiện thiết kế lớp RootDetectionHelper như sau

- Phát hiện firmware được xây dựng dành cho nhà phát triển hoặc đã chỉnh sửa, thông qua Build.TAGS
- Phát hiện các ứng dụng liên quan đến root đã được cài đặt
- Tìm kiếm các tệp nhị phân (su) hoặc tệp liên quan đến root trong hệ thống

Cụ thể các hàm bên trong lớp sẽ như sau

- Hàm **detectDeveloperBuild()**


```
private fun detectDeveloperBuild(): Boolean {
    return Build.TAGS?.contains("test-keys") == true
}
```

- Nếu Build.TAGS có chứa "test-keys", thì thiết bị có khả năng đã bị root

- Hàm **detectRootedAPKs()**

```
private fun detectRootedAPKs(context: Context): Boolean {
    val knownRootedAPKs = listOf(
        "com.noshufou.android.su",
        "eu.chainfire.supersu",
        "com.koushikdutta.superuser"
    )
    val pm = context.packageManager
    return knownRootedAPKs.any { packageName →
        try {
            pm.getPackageInfo(packageName, 0)
            true
        } catch (e: Exception) {
            false
        }
    }
}
```

- Dùng PackageManager để kiểm tra sự tồn tại của các gói APK phổ biến liên quan đến root

- Hàm **detectForSUBinaries()**

```
private fun detectForSUBinaries(): Boolean {  
    val suPaths = listOf(  
        "/system/bin/su",  
        "/system/xbin/su",  
        "/system/app/Superuser.apk",  
        "/system/xbin/daemonsu"  
    )  
    return suPaths.any { path →  
        File(path).exists()  
    }  
}
```

- Kiểm tra sự tồn tại của các đường dẫn phổ biến liên quan đến root

- Hàm **isDeviceRooted()**

```
fun isDeviceRooted(applicationContext: Context?): Boolean {  
    if (applicationContext == null) return false // Trả về false nếu context null  
    return detectDeveloperBuild() || detectRootedAPKs(applicationContext) || detectForSUBinaries()  
}
```

- Kiểm tra lần lượt 3 hàm ở trên để kiểm tra thiết bị có bị Root hay không

c. Tác dụng:

Sau khi thiết kế lớp RootDetectionHelper thì ta sẽ bảo vệ ứng dụng khỏi các thiết bị đã thực hiện root sử dụng đến, từ đó đảm bảo

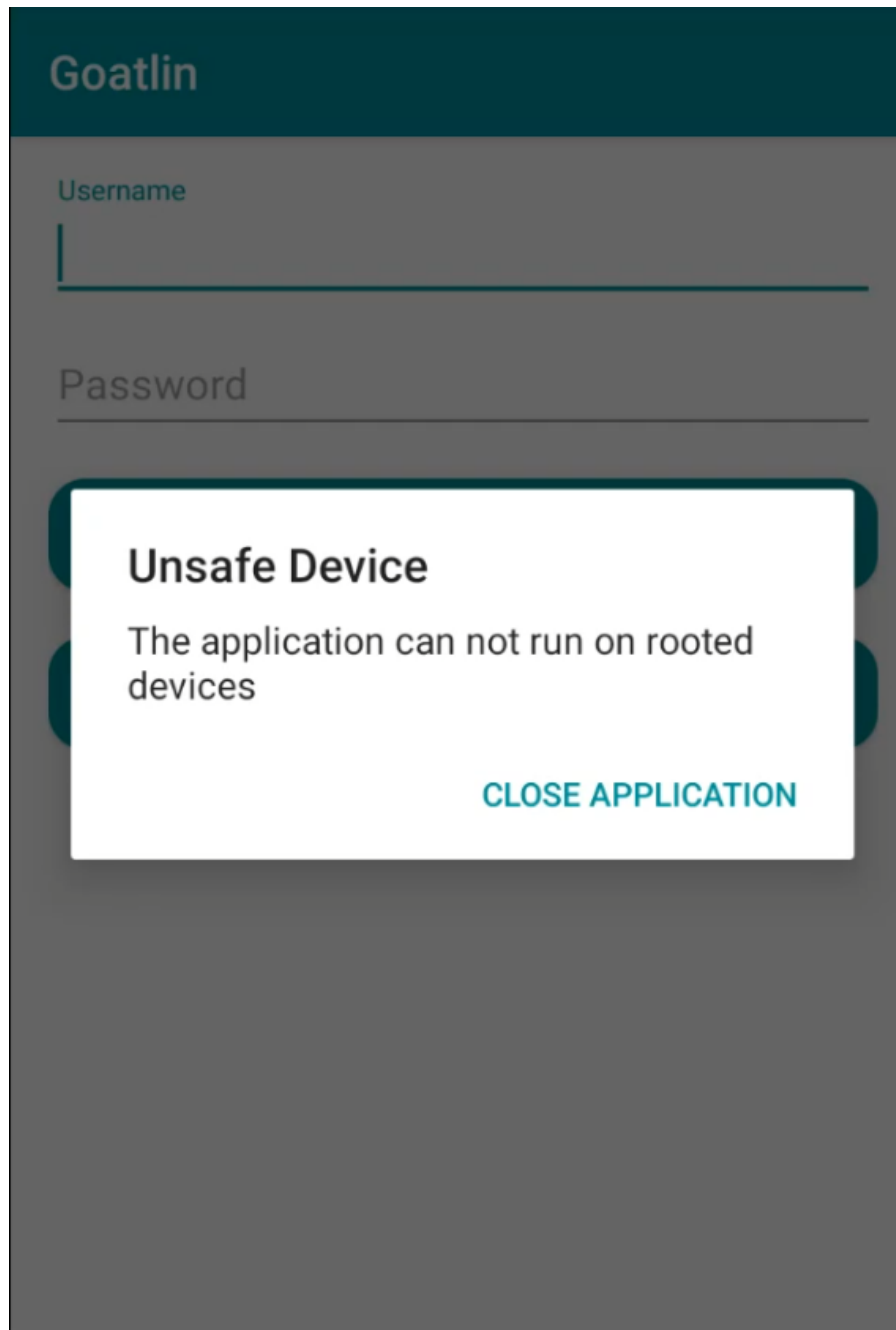
- **Bảo vệ dữ liệu và bảo mật:** Thiết bị đã root có thể dễ dàng bị tấn công bởi phần mềm độc hại hoặc các ứng dụng không an toàn. Bằng cách ngăn cản thiết bị root, ta giảm thiểu nguy cơ dữ liệu nhạy cảm bị đánh cắp hoặc bị lạm dụng

- **Đảm bảo tính toàn vẹn của ứng dụng:** Root có thể cho phép người dùng thay đổi hoặc can thiệp vào mã nguồn của ứng dụng, dẫn đến các hành vi không mong muốn hoặc không dự đoán trước. Ngăn cản thiết bị root giúp bảo vệ tính toàn vẹn và hoạt động ổn định của ứng dụng

- **Bảo vệ trải nghiệm người dùng:** Thiết bị root có thể gây ra các vấn đề về hiệu suất và ổn định. Bằng cách ngăn cản thiết bị root, ta đảm bảo rằng người dùng có trải nghiệm tốt nhất khi sử dụng ứng dụng

d.Kết quả:

Khi thực hiện tiến vào ứng dụng thì trước tiên nhất, ứng dụng sẽ thực hiện kiểm tra thiết bị, nếu nó đã bị root thì sẽ bị bắt buộc thoát ra khỏi ứng dụng, còn nếu chưa root thì thiết bị sẽ tiếp tục sử dụng ứng dụng bình thường



3.2.3. M1: Improper Platform Usage (Sử dụng sai nền tảng)

a. Phân tích

Mô tả lỗ hổng: Lỗi Improper Platform Usage trong ứng dụng Goatlin xuất hiện do việc cấu hình sai các Content Providers, một thành phần quan trọng trong hệ thống Android để chia sẻ dữ liệu giữa các ứng dụng. Dưới đây là các điểm yếu chính:

- Trong file AndroidManifest, AccountProvider được khai báo với thuộc tính exported="true", cho phép mọi ứng dụng trên thiết bị truy cập vào.

```
<provider
    android:name="com.cx.goatlin.AccountProvider"
    android:authorities="com.cx.goatlin.accounts"
    android:enabled="true"
    android:exported="true" />
```

- Code trong lớp AccountProvider.kt cho thấy Content URI được định nghĩa như sau:

```
companion object {
    private val AUTHORITY = "com.cx.goatlin.accounts"
    private val ACCOUNTS_TABLE = "Accounts"
    val CONTENT_URI : Uri = Uri.parse(uriString: "content://" + AUTHORITY + "/" +
        ACCOUNTS_TABLE)
    private val DATABASE_NAME = "data"
}
```

- Tương tự, NotesProvider cũng được khai báo với exported="true", dẫn đến nguy cơ rò rỉ dữ liệu từ các ghi chú của người dùng. Dữ liệu có thể bị truy vấn thông qua Content URI được định nghĩa trong lớp NotesProvider:

```
<provider
    android:name="com.cx.goatlin.NotesProvider"
    android:authorities="com.cx.goatlin.notes"
    android:enabled="true"
    android:exported="true"></provider>
```

Mức độ ảnh hưởng của lỗ hổng: Kẻ tấn công có thể sử dụng công cụ adb để thực hiện truy vấn hoặc chèn dữ liệu trái phép vào Content Provider này:

- Truy vấn dữ liệu:

```
D:\Android>adb shell content query --uri content://com.cx.goatlin.accounts/Accounts
Row: 0 id=1, username=admin, password=12345678
Row: 1 id=2, username=, password=
Row: 2 id=3, username=huyho, password=huyho
```

- Chèn dữ liệu trái phép: chèn account huyho123 mà không cần Signup

```
D:\Android>adb shell content insert --uri content://com.cx.goatlin.accounts/Accounts --bind username:s:huyho123 --bind password:s:huyho123

D:\Android>adb shell content query --uri content://com.cx.goatlin.accounts/Accounts
Row: 0 id=1, username=admin, password=12345678
Row: 1 id=2, username=, password=
Row: 2 id=3, username=huyho, password=huyho
Row: 3 id=4, username=huyho123, password=huyho123
```

- Truy vấn dữ liệu:

```
D:\Android>adb shell content query --uri content://com.cx.goatlin.notes/Notes
Row: 0 id=1, title=whvw, content=whvw, createdAt=2024-12-11 15:37:00, owner=3
Row: 1 id=2, title=whvw2, content=gijkmn, createdAt=2024-12-11 15:37:24, owner=3
Row: 2 id=3, title=whvw, content=kxb, createdAt=2024-12-11 16:18:56, owner=4
Row: 3 id=4, title=whvw2, content=kxb, createdAt=2024-12-11 16:19:02, owner=4
D:\Android>
```

b. Khắc phục

Để khắc phục lỗ hổng này, cải tiến sau được thực hiện: Cấu hình lại Content Providers: Đặt thuộc tính exported về false để ngăn chặn truy cập từ các ứng dụng bên ngoài (Chỉ đặt true cho mỗi module Login, còn lại là false).

```

<provider
    android:name="com.cx.goatlin.AccountProvider"
    android:authorities="com.cx.goatlin.accounts"
    android:enabled="true"
    android:exported="false" />
<provider
    android:name="com.cx.goatlin.NotesProvider"
    android:authorities="com.cx.goatlin.notes"
    android:enabled="true"
    android:exported="false"></provider>

```

c. Tác dụng

- Việc đặt exported="false" giúp ngăn chặn các ứng dụng khác truy cập trái phép.
- Loại bỏ nguy cơ rò rỉ thông tin đăng nhập (username, password).
- Ngăn chặn việc chèn dữ liệu trái phép, đảm bảo tính toàn vẹn của hệ thống.
- Tăng cường khả năng mở rộng với cấu hình bảo mật chặt chẽ.

d. Kết quả

Kể từ công không thể nào truy vấn được nữa, hệ thống sẽ báo lỗi và ngăn chặn việc này ngay lập tức:

```

C:\Users\PC>adb shell content insert --uri content://com.c
x.goatlin.accounts/Accounts --bind username:s:huyho --bin
d password:s:huyho
Error while accessing provider:com.cx.goatlin.accounts
java.lang.SecurityException: Permission Denial: opening pr
ovider com.cx.goatlin.AccountProvider from (null) (pid=533
0, uid=2000) that is not exported from UID 10083
    at android.os.Parcel.readException(Parcel.java:194
2)
    at android.os.Parcel.readException(Parcel.java:188
8)
    at android.app.IActivityManager$Stub$Proxy.getCont
entProviderExternal(IActivityManager.java:7137)
    at com.android.commands.content.Content$Command.ex
ecute(Content.java:441)
    at com.android.commands.content.Content.main(Conte
nt.java:664)
    at com.android.internal.os.RuntimeInit.nativeFinis
hInit(Native Method)
    at com.android.internal.os.RuntimeInit.main(Runtim
eInit.java:284)

```

3.2.4. M2: Insecure Data Storage (Lưu trữ dữ liệu không an toàn)

a. Phân tích:

Mô tả lỗ hổng: Dưới đây là hàm createAccount trong class DatabaseHelper dùng để tạo 1 tài khoản và lưu vào local: /data/data/com.cx.vulnerablekotlinapp/databases/data Có thể thấy, hàm này lưu luôn plaintext vào csdl, không thông qua phương thức mã hoá nào để bảo vệ tài khoản user.

```
public fun createAccount(username: String, password: String) : Boolean {
    val db: SQLiteDatabase = this.writableDatabase
    val record: ContentValues = ContentValues()
    var status = true

    record.put("username", username)
    record.put("password", password)

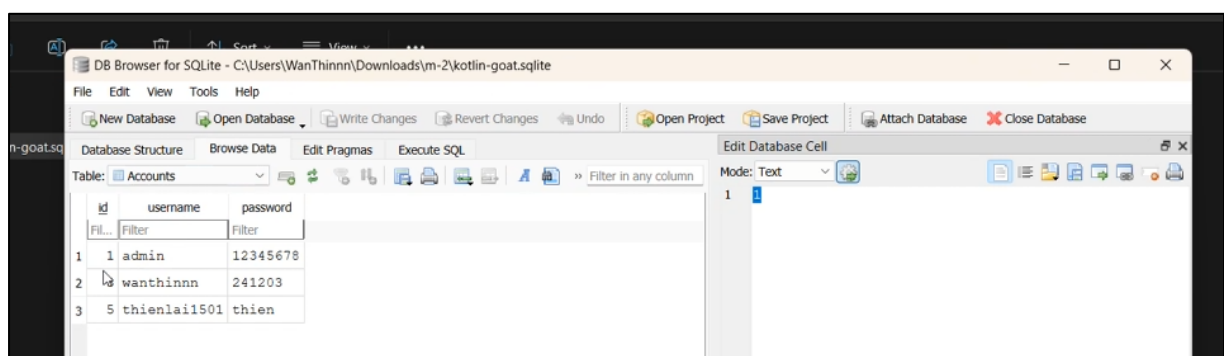
    try {
        db.insertOrThrow(TABLE_ACCOUNTS, nullColumnHack: null, record)
    }
    catch (e: SQLException) {
        Log.e(tag: "Database signup", e.toString())
        status = false
    }
    finally {
        return status
    }
}
```

Mức độ ảnh hưởng của lỗ hổng

- Ta sử dụng adb để pull file data base về máy

```
C:\Users\WanThinnn>adb -s emulator-5556 pull /data/data/com.cx.vulnerablekotlinapp/databases/data "C:\Users\WanThinnn\Downloads\m-2\kot
lin-goat.sqlite"
/data/data/com.cx.vulnerablekotlinapp/databases/data: 1 file pulled. 12.1 MB/s (28672 bytes in 0.002s)
```

- Và ta dễ dàng xem được dữ liệu được lưu => rất nguy hiểm



b. Khắc phục:

Ta nên sử dụng một số thuật toán mã hoá 1 chiều mạnh mẽ như SHA3, SHA2, Bcrypt để hash mật khẩu, khi lưu vào csdl, dù có bị lộ ra thì mật khẩu vẫn an toàn

Nhóm sử dụng thuật toán Bcrypt được cải tiến với SHA512 và lưu được mật khẩu dài hơn 72 byte so với Bcrypt truyền thống.

- Dưới đây là thuật toán Bcrypt được cải tiến trong class CryptoHelper

```
// Mã hóa mật khẩu bằng Bcrypt
fun encryptpw(original: String): String {
    return BCrypt.with(LongPasswordStrategies.hashSha512(BCrypt.Version.VERSION_2A)).hashToString(cost: 6, original.toCharArray());
}

// Kiểm tra mật khẩu với Bcrypt
fun verifypw(password: String, hashed: String): Boolean {
    return BCrypt.verifier().verify(password.toCharArray(), hashed).verified
}
```

- Khi tạo 1 tài khoản mới, password sẽ được mã hoá và lưu vào CSDL:

```
public fun createAccount(name: String, username: String, password: String): Boolean {
    val db: SQLiteDatabase = this.writableDatabase
    val record = ContentValues().apply {
        put("name", name)
        put("username", username)
        put("password", CryptoHelper.encryptpw(password))
    }

    return try {
        db.insertOrThrow(TABLE_ACCOUNTS, nullColumnHack: null, record)
        true
    } catch (e: SQLException) {
        Log.e(tag: "Database signup", msg: "Error creating account: ${e.message}")
        false
    }
}
```

c. Tác dụng:

Việc áp dụng mã hóa mật khẩu trong quá trình lưu trữ giúp tăng cường đáng kể tính bảo mật, với những lợi ích cụ thể như sau:

- **Bảo vệ dữ liệu khi bị tấn công:** Dù cơ sở dữ liệu bị truy cập trái phép, kẻ tấn công cũng không thể lấy được mật khẩu thực, do các mật khẩu đã được hash.

- **Kháng tấn công brute force:** Các thuật toán như Bcrypt được thiết kế để chậm, làm giảm tốc độ của các cuộc tấn công dò mật khẩu.

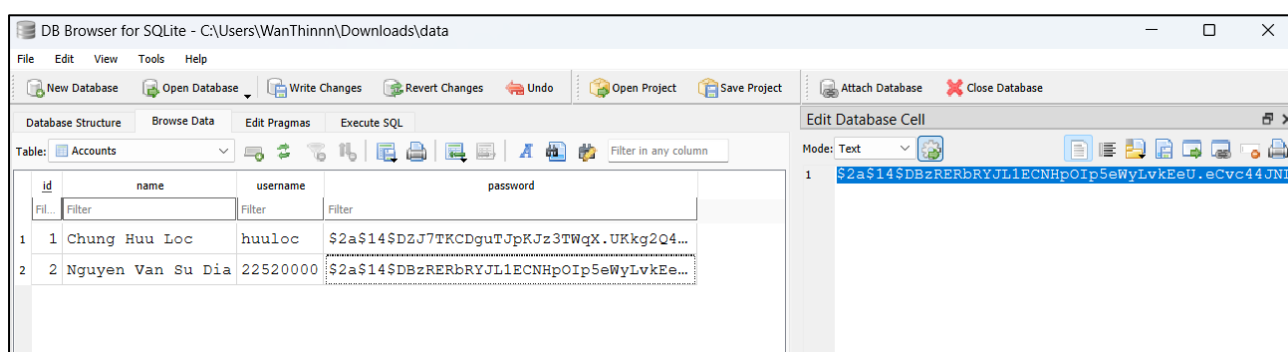
- **Tính an toàn trước rò rỉ:** Hashing mật khẩu ngăn chặn khả năng sử dụng lại thông tin đăng nhập trên các hệ thống khác, nếu mật khẩu bị lộ.

- **Tương thích tốt:** Việc áp dụng Bcrypt kết hợp SHA512 đảm bảo tính tương thích cao và khả năng xử lý tốt với các mật khẩu dài hơn 72 byte so với Bcrypt truyền thống, phù hợp với các yêu cầu hiện đại.

Nhờ áp dụng các thuật toán hash mạnh như Bcrypt, mật khẩu người dùng sẽ được bảo vệ tốt hơn, giảm thiểu rủi ro bị lộ thông tin nhạy cảm.

d. Kết quả

Khi ta xem CSDL ở local thì sẽ không biết được password:



The screenshot shows the DB Browser for SQLite interface. The 'Accounts' table is selected, and its data is displayed in a table view. The 'password' column contains hashed values, demonstrating that the passwords are not stored in plain text.

	id	name	username	password
1	1	Chung Huu Loc	huuloc	\$2a\$14\$DzJ7TKCDguTJpKJz3TWqX.UKkg2Q4...
2	2	Nguyen Van Su Dia	22520000	\$2a\$14\$DBzRERbRYJL1ECNHpOIpSeWyLvKEe...

3.2.5. M3: Insecure Communication (Giao tiếp không an toàn)

a. Phân tích:

Phần lỗ hổng được xác định:

```
fun create(): Client {
    val hostname: String = PreferenceHelper.getString("ip_address", "127.0.0.1")
    val port: String = PreferenceHelper.getString("port", "8080")
    val baseUrl: String = "http://${hostname}:${port}"

    val retrofit = Retrofit.Builder()
        .addCallAdapterFactory(RxJava2CallAdapterFactory.create())
        .addConverterFactory(GsonConverterFactory.create())
        .baseUrl(baseUrl)
        .build()

    return retrofit.create(Client::class.java)
}
```

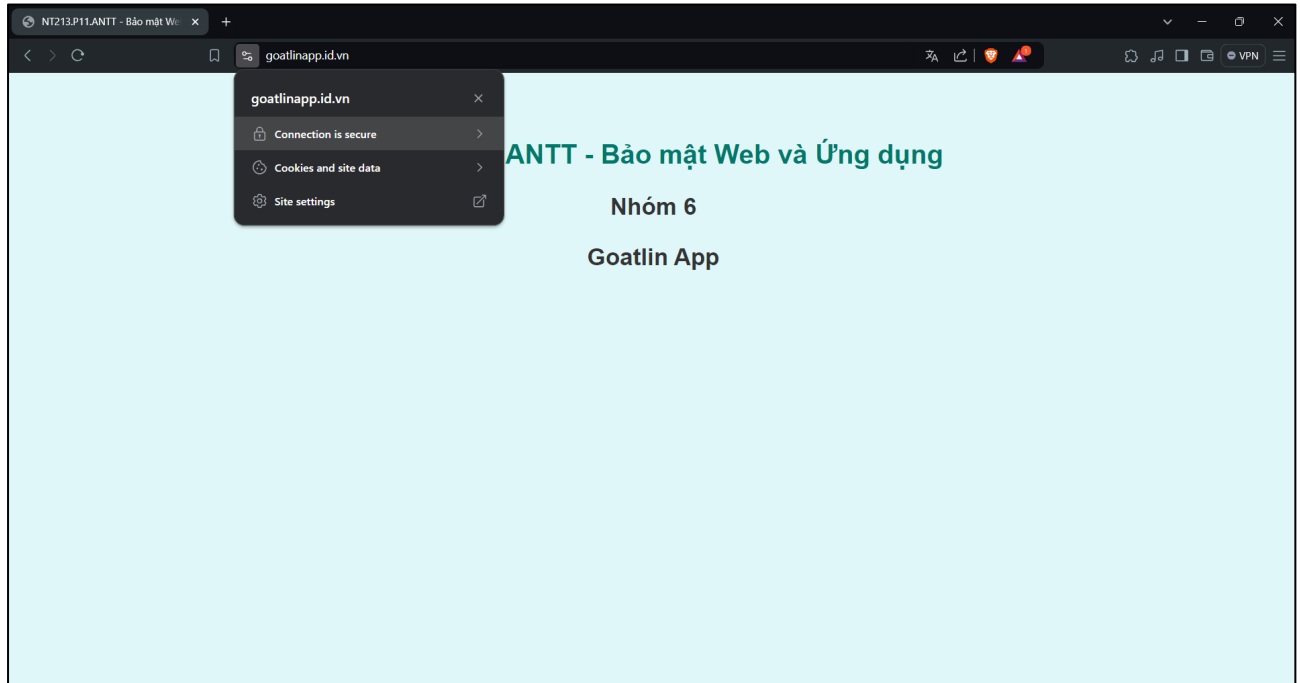
Mô tả lỗ hổng: Insecure Communication đề cập đến các lỗ hổng hoặc thiếu sót trong cách các ứng dụng di động truyền tải dữ liệu, dẫn đến nguy cơ bị đánh cắp hoặc can thiệp thông tin nhạy cảm. Cụ thể các lỗ hổng sẽ bao gồm

- Dữ liệu không được mã hóa: Truyền dữ liệu qua HTTP thay vì HTTPS
- Sử dụng giao thức mã hóa yếu: Sử dụng SSL/TLS phiên bản lỗi thời hoặc không cấu hình đúng cách, thiếu xác thực chứng chỉ máy chủ, dẫn đến tấn công man-in-the-middle (MitM)
- Quản lý phiên không an toàn: Cookie hoặc token truy cập không được bảo vệ trong khi truyền

Mức độ ảnh hưởng của lỗ hổng: rất nghiêm trọng khi mà dữ liệu có thể bị đánh cắp hoặc bị theo dõi, thay đổi hoặc chặn mất

b. Khắc phục:

Tạo 1 trang web an toàn HTTPS để thực hiện truyền dữ liệu, từ đó đảm bảo an toàn cho dữ liệu trong quá trình nó được truyền tải



Lấy 2 chứng chỉ là server.key và server.crt của trang web để thực hiện sử dụng Certificate Pinning để giúp ngăn chặn các cuộc tấn công Man-in-the-Middle (MitM), vì ứng dụng chỉ chấp nhận chứng chỉ của máy chủ có mã băm SHA256 đã xác định

- Ta sẽ lấy mã băm bằng câu lệnh: **openssl x509 -in server.crt -pubkey -noout | openssl pkey -pubin -outform der | openssl dgst -sha256 -binary | openssl enc -base64**

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26120.1252]
(c) Microsoft Corporation. All rights reserved.

C:\Code\Bảo mật web và ứng dụng - NT213.P11.ANTT\an\doan\Goatlin\packages\clients\android\ssl>openssl x509 -in server.crt -pubkey -noout | openssl pkey -pubin -outform der | openssl dgst -sha256 -binary | openssl enc -base64
DLWMRNpw7oVgjZM40yQNLf6lx14FxqeH2z+gzXevsi8=
```

- Sửa đổi Phương thức tạo của API, thông qua OkHttpClient để quản lí các yêu cầu mạng

```
fun create(): Client {
    val certificatePinner = CertificatePinner.Builder()
        .add( pattern: "goatlinapp.id.vn:8443", ...pins: "sha256/DLwMRNpw7oVgjZM40yQNLf61x14FxqeH2z+gzXevsi8=")
        .build()

    val client: OkHttpClient = OkHttpClient.Builder()
        .certificatePinner(certificatePinner)
        .build()

    val retrofit = Retrofit.Builder()
        .addCallAdapterFactory(RxJava2CallAdapterFactory.create())
        .addConverterFactory(GsonConverterFactory.create())
        .baseUrl("https://goatlinapp.id.vn:8443")
        .client(client)
        .build()

    return retrofit.create(Client::class.java)
}
```

+ certificatePinner: cấu hình Certificate Pinning để bảo vệ khỏi các cuộc tấn công giả mạo

- CertificatePinner được sử dụng để xác thực chứng chỉ của máy chủ theo cách mà chỉ các chứng chỉ với mã băm SHA-256 cụ thể được chấp nhận. Nếu chứng chỉ không khớp, kết nối sẽ bị từ chối

+ OkHttpClient: tạo một OkHttpClient với Certificate Pinning

- OkHttpClient.Builder() được sử dụng để tạo một client HTTP tùy chỉnh
- .certificatePinner(certificatePinner) gán Certificate Pinner vào client HTTP, đảm bảo rằng mỗi kết nối HTTP được tạo ra sẽ kiểm tra chứng chỉ của máy chủ xem có khớp với chứng chỉ pin hay không

+ Retrofit: cấu hình Retrofit để sử dụng OkHttpClient tùy chỉnh và giao tiếp với máy chủ HTTPS

- Retrofit là một thư viện được sử dụng để gửi yêu cầu HTTP từ ứng dụng Android
- .addCallAdapterFactory(RxJava2CallAdapterFactory.create()) cho phép sử dụng RxJava để xử lý các yêu cầu bất đồng bộ (asynchronous requests)
- .addConverterFactory(GsonConverterFactory.create()) chỉ định cách chuyển đổi dữ liệu JSON thành đối tượng Java (hoặc Kotlin) sử dụng thư viện **Gson**.
- .baseUrl("https://goatlinapp.id.vn:8443") đặt URL cơ sở cho các yêu cầu HTTP. Tất cả các URL sau đó sẽ được xây dựng dựa trên URL cơ sở này.

- .client(client) chỉ định client đã được cấu hình với Certificate Pinner và sẽ được sử dụng trong Retrofit để thực hiện các yêu cầu mạng

c. Tác dụng:

Thông qua các cài đặt để đảm bảo dữ liệu được truyền qua giao thức https thì nhóm đã mang lại nhiều lợi ích quan trọng bao gồm:

- **Bảo vệ dữ liệu người dùng:** Sử dụng các giao thức bảo mật như SSL/TLS giúp bảo vệ dữ liệu nhạy cảm khỏi bị chặn và đánh cắp trong quá trình truyền tải

- **Ngăn chặn tấn công Man-in-the-Middle (MITM):** Bằng cách đảm bảo rằng các kết nối được mã hóa và xác thực đúng cách, ta có thể ngăn chặn kẻ tấn công chèn mình vào giữa quá trình giao tiếp

- **Bảo vệ khỏi phần mềm độc hại:** Việc mã hóa dữ liệu và sử dụng các giao thức bảo mật giúp ngăn chặn phần mềm độc hại xâm nhập và lây lan qua các kênh giao tiếp không an toàn

d. Kết quả:

Khi ta sử dụng WireShark để bắt gói tin truyền dữ liệu của ứng dụng thì dữ liệu được gửi đi đã được mã hóa, thứ WireShark bắt được là quy trình bắt tay 3 bước giữa server và client

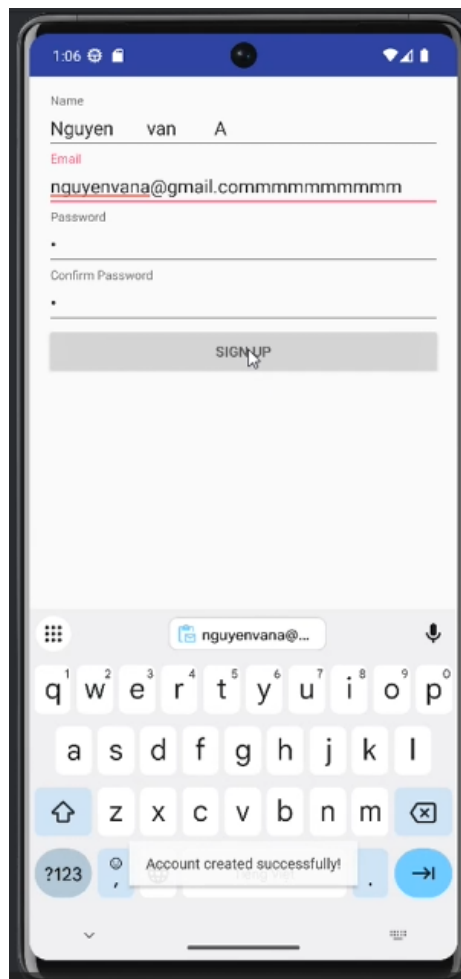
No.	Time	Source	Destination	Proto	Length	Info
1020	18.956348	192.168.1.147	13.211.53.104	TCP	66	61367 → 8443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
1030	19.090806	13.211.53.104	192.168.1.147	TCP	66	8443 → 61367 [SYN, ACK] Seq=0 Ack=1 Win=62727 Len=0 MSS=1452 SACK_PERM WS=128
1031	19.090956	192.168.1.147	13.211.53.104	TCP	54	61367 → 8443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
1032	19.093277	192.168.1.147	13.211.53.104	TLS	235	Client Hello (SNI=goatlinapp.id.vn)
1042	19.226722	13.211.53.104	192.168.1.147	TCP	60	8443 → 61367 [ACK] Seq=1 Ack=182 Win=62592 Len=0
1043	19.226767	13.211.53.104	192.168.1.147	TLS	1506	Server Hello
1044	19.226826	13.211.53.104	192.168.1.147	TLS	853	Certificate, Server Key Exchange, Server Hello Done
1045	19.226860	192.168.1.147	13.211.53.104	TCP	54	61367 → 8443 [ACK] Seq=182 Ack=2252 Win=65280 Len=0
1047	19.243528	192.168.1.147	13.211.53.104	TLS	147	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message
1049	19.263529	192.168.1.147	13.211.53.104	TLS	351	Application Data
1052	19.377691	13.211.53.104	192.168.1.147	TLS	344	New Session Ticket, Change Cipher Spec, Encrypted Handshake Message
1053	19.400424	13.211.53.104	192.168.1.147	TLS	255	Application Data
1054	19.400484	192.168.1.147	13.211.53.104	TCP	54	61367 → 8443 [ACK] Seq=572 Ack=2743 Win=65024 Len=0

3.2.6. M4: Insecure Authentication (Xác thực không an toàn)

a. Phân tích:

Mô tả lỗ hổng: Khi ta tạo 1 tài khoản, app không có bất kỳ sự ràng buộc nào về mật khẩu, tên người dùng và cả họ tên của họ

Mức độ ảnh hưởng của lỗ hổng: Hình bên dưới cho thấy, khi ta tạo 1 tài khoản với mật khẩu 1 ký tự, email không phù hợp và họ tên thì không được chuẩn hoá. Tuy nhiên app vẫn báo tạo thành công tài khoản



b. Khắc phục:

Ta tiến hành viết 1 class CheckStringHelper có nhiều ràng buộc để kiểm tra plaintext người dùng nhập vào.

```
1 package com.cx.goatlin.helpers
2
3 class CheckStringHelper {
4     companion object {
5         // Hàm kiểm tra username
6         > fun validateName(input: String): Boolean {...}
7
8         // Hàm viết hoa chữ cái đầu
9         fun capitalizeFirstLetter(input: String): String
10        {...}
11
12        // Hàm kiểm tra khoảng trắng và ký tự '-'
13        > fun validateNoWhitespaceOrDash(input: String): Boolean {...}
14    }
15 }
```

Về Họ tên, sử dụng hàm chuẩn hoá, đảm bảo người dùng không nhập dư thừa ký tự khoảng trắng hoặc ký tự lạ.

```
// Kiểm tra và chuẩn hóa tên
if (!CheckStringHelper.validateName(name)) {
    this.name.error = "Invalid name. Only letters and one space between words allowed."
    this.name.requestFocus()
    return
}
else{
    name = CheckStringHelper.capitalizeFirstLetter(name)
}
```

The screenshot shows a form with two input fields. The first field, labeled 'Full name', contains the text 'Nguyen Van B'. A red error icon is visible to the right of this field. The second field, labeled 'Username', contains the text 'b # jd ddd dd'. A tooltip message is displayed over the 'Full name' field, stating 'Invalid name. Only letters and one space between words allowed.'

Về Email, nhóm sẽ đổi sang Username, không sử dụng email vì dữ liệu chỉ lưu ở local, việc sử dụng email thì cần phải thêm bước xác thực email đó, ngoài ra thêm các ràng buộc như: không được sử dụng khoảng trắng, không có ký tự lạ,...

```

if (!CheckStringHelper.validateNoWhitespaceOrDash(username)){
    this.email.error = "Invalid username. No whitespace or '-' allowed."
    this.email.requestFocus()
    return
}

```

The screenshot shows a web form with two input fields: "Username" and "Password". The "Username" field contains the text "nguyen van b". A red error message box is displayed over the "Username" field, containing the text "Invalid username. No whitespace or '-' allowed." A red exclamation mark icon is visible in the top right corner of the "Username" field.

Về Mật khẩu, sử dụng Bcrypt hash và lưu vào CSDL (phần 3.2.4 có đề cập) và đồng thời thêm hàm PasswordHelper để kiểm tra độ an toàn của mật khẩu

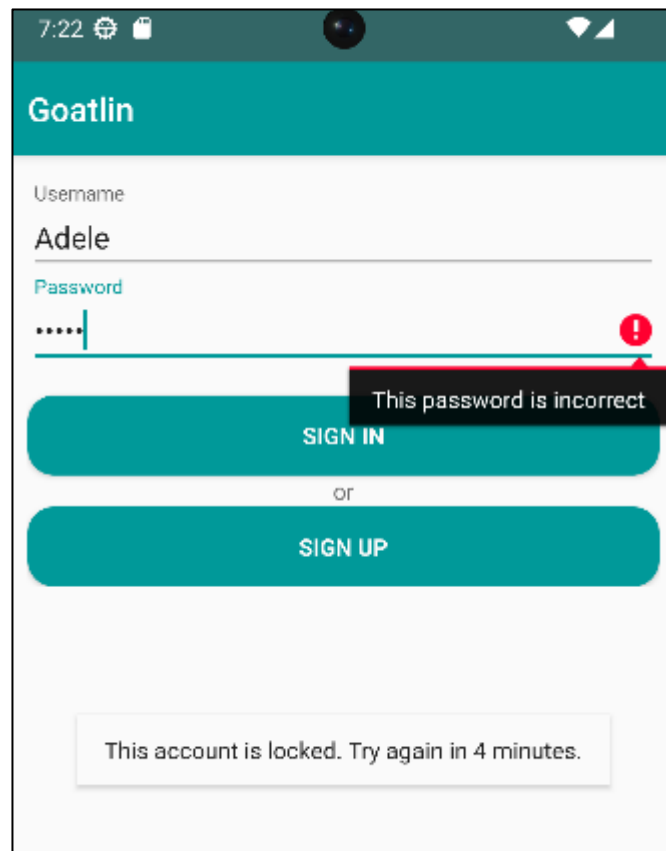
```

if (!PasswordHelper.strength(password)) {
    this.password.error = ""|Weak password. Please use:
        |* both upper and lower case letters
        |* numbers
        |* special characters (e.g. !"#$%&')
        |* from 10 to 128 characters sequence""|.trimMargin()
    this.password.requestFocus()
    return;
}

```

The screenshot shows a web form with two input fields: "Password" and "Confirm Password". The "Password" field is empty. A red error message box is displayed over the "Password" field, containing the text "Weak password. Please use:" followed by a list of requirements: "* both upper and lower case letters", "* numbers", "* special characters (e.g. !"#\$%&')", and "* from 10 to 128 characters sequence". A red exclamation mark icon is visible in the top right corner of the "Password" field.

Ngoài ra, ứng dụng cũng kiểm tra xem việc User đăng nhập sai quá 05 lần hay không? Nếu có thì sẽ chặn User đó trong vòng 5 phút.



c. Tác dụng

Cải thiện xác thực không an toàn trong ứng dụng di động không chỉ nâng cao trải nghiệm người dùng mà còn giảm thiểu nguy cơ bị tấn công:

- **Cải thiện chất lượng dữ liệu đầu vào:** Việc chuẩn hóa họ tên giúp loại bỏ các ký tự dư thừa và lạ, đảm bảo tính thống nhất và dễ quản lý dữ liệu người dùng.

- **Giảm nguy cơ sử dụng thông tin không xác thực:** Bằng cách chuyển từ sử dụng email sang username, ứng dụng tránh được việc lưu trữ các thông tin nhạy cảm như email mà không có hệ thống xác thực phù hợp. Điều này giảm thiểu khả năng khai thác dữ liệu cá nhân nếu xảy ra vi phạm.

- **Bảo vệ thông tin đăng nhập:**

- + Việc sử dụng hàm kiểm tra độ mạnh của mật khẩu (PasswordHelper) giúp ngăn chặn các mật khẩu yếu, dễ bị tấn công brute force hoặc dictionary attack.

- + Hash mật khẩu bằng Bcrypt đảm bảo rằng ngay cả khi cơ sở dữ liệu bị rò rỉ, mật khẩu thực tế vẫn khó bị giải mã.

- **Tăng độ tin cậy của ứng dụng:** Các ràng buộc và quy chuẩn trong quá trình xác thực không chỉ giúp ứng dụng an toàn hơn mà còn tạo ra ấn tượng chuyên nghiệp và đáng tin cậy đối với người dùng.

d. Kết quả

(Đã trình bày chung với phần **b. Khắc phục**)

3.2.7. M5: Insufficient Cryptography (Mật mã học không đủ mạnh)

a. Phân tích:

Mô tả lỗ hổng: Chúng ta xem xét Class CryptoHelper:

```
class CryptoHelper {
    companion object {
        fun encrypt(original: String): String {
            var encrypted: String = ""

            for (c in original) {
                val ascii: Int = c.toInt()
                val lowerBoundary: Int = if (c.isUpperCase()) 65 else 97

                if (ascii in 65..90 || ascii in 97..122) {
                    encrypted += ((ascii + SHIFT - lowerBoundary) % 26 + lowerBoundary).toChar()
                } else {
                    encrypted += c
                }
            }

            return encrypted
        }

        fun decrypt(encrypted: String): String {
            var original: String = ""

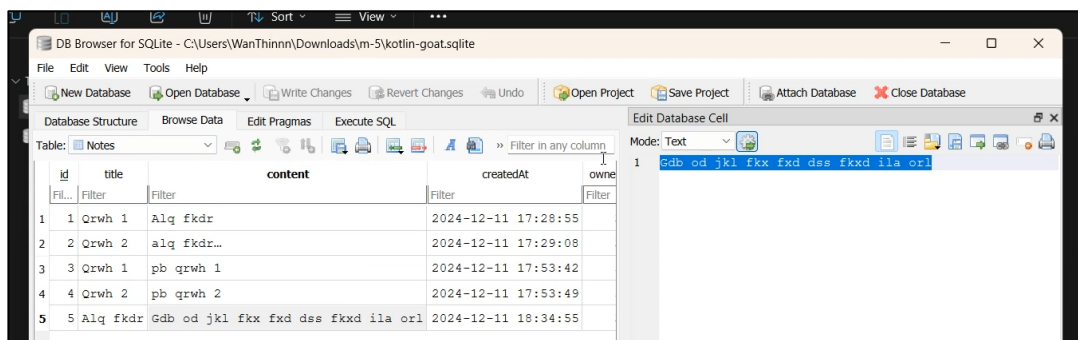
            for (c in encrypted) {
                val ascii: Int = c.toInt()
                val lowerBoundary: Int = if (c.isUpperCase()) 65 else 97

                if (ascii in 65..90 || ascii in 97..122) {
                    original += ((ascii - SHIFT - lowerBoundary) % 26 + lowerBoundary).toChar()
                } else {
                    original += c
                }
            }

            return original
        }
    }
}
```

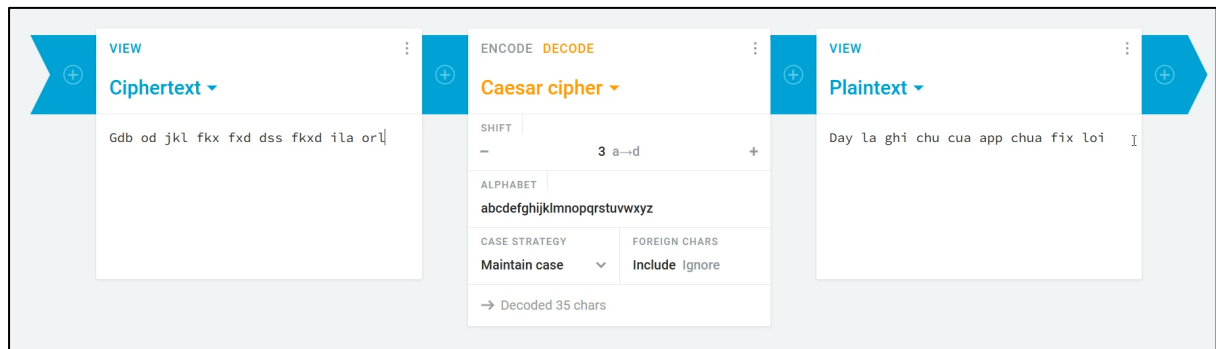
Mức độ ảnh hưởng của lỗ hổng: Có thể thấy, app sử dụng thuật toán mã hoá Caesar, đây là một thuật toán mật mã học cổ điển, và không còn an toàn ở 2024 nữa.

- Ta xem CSDL:



	id	title	content	createdAt	owner
1	1	Qrwh 1	Alq fkdr	2024-12-11 17:28:55	
2	2	Qrwh 2	alq fkdr...	2024-12-11 17:29:08	
3	3	Qrwh 1	pb qrwh 1	2024-12-11 17:53:42	
4	4	Qrwh 2	pb qrwh 2	2024-12-11 17:53:49	
5	5	Alq fkdr	Gdb od jkl fxx fxd dss fxxd ila orl	2024-12-11 18:34:55	

- Sử dụng tool online để Decrypt:



b. Khắc phục:

Về việc mã hoá CSDL, ta sẽ sử dụng thuật toán mạnh mẽ như AES-GCM-256. Tuy nhiên, việc lưu khoá AES ở đâu thì cần phải giải quyết an toàn nhất. Ở đây, nhóm sử dụng 4 module chính sau để giải quyết bài toán này:

```
3 package com.cx.goatlin.helpers
4 > import ...
15
16
17 class CryptoHelper {
18     companion object {
19         private const val IV_LENGTH = 12
20         // Tạo khóa người dùng trong Android Keystore
21 > fun createUserKey(username: String) {...}
52
53         // Thêm hàm kiểm tra khóa tồn tại
54 > private fun ensureKeyExists(username: String) {...}
61
62         // Lấy khóa người dùng từ Android Keystore
63 > private fun getUserKey(username: String): SecretKey? {...}
68
69         // Mã hóa dữ liệu bằng khóa người dùng từ Keystore
70 > fun encrypt(original: String, username: String): String {...}
86
87         // Giải mã dữ liệu bằng khóa người dùng từ Keystore
88 > fun decrypt(message: String, username: String): String {...}
113
```

1. `createUserKey(username: String)`: Tạo một khóa AES cho người dùng mới (dựa trên tên người dùng) và lưu khóa đó vào **Android Keystore**. Nếu khóa đã tồn tại, nó sẽ không tạo lại. Vì mỗi người dùng chỉ có 1 username duy nhất, nên mỗi khóa của họ cũng được đảm bảo là duy nhất.
2. `ensureKeyExists(username: String)`: Kiểm tra xem khóa của người dùng có tồn tại trong Keystore không. Nếu không, sẽ gọi phương thức `createUserKey` để tạo khóa mới.

3. `getUserKey(username: String)`: Lấy khóa của người dùng từ **Android Keystore**.
4. `encrypt(original: String, username: String)`: Mã hóa chuỗi dữ liệu (original) bằng khóa người dùng trong Keystore và sử dụng thuật toán AES/GCM/NoPadding. Dữ liệu mã hóa trả về được mã hóa Base64.
5. `decrypt(message: String, username: String)`: Giải mã dữ liệu đã mã hóa (dưới dạng Base64) bằng khóa của người dùng trong Keystore và trả về chuỗi gốc.

Cách hoạt động:

- **Keystore** lưu trữ các khóa bảo mật mà không để lộ chúng cho ứng dụng.
- Dữ liệu được mã hóa bằng **AES** trong chế độ **GCM**
- Phương thức `encrypt` tạo IV ngẫu nhiên cho mỗi lần mã hóa để bảo mật cao hơn. IV được lưu kèm với dữ liệu mã hóa.
- Phương thức `decrypt` để giải mã dữ liệu được load lên từ CSDL.

(Nhắc lại)

Về Android Keystore: **Android Keystore** là một **phần mềm** được tích hợp trong hệ điều hành Android, cho phép các ứng dụng bảo mật lưu trữ và sử dụng các khóa mã hóa mà không cần phải tiết lộ các khóa này cho ứng dụng hoặc hệ điều hành.

Cách hoạt động của Android Keystore:

- **Lưu trữ khóa an toàn**: Android Keystore đảm bảo rằng các khóa mã hóa được tạo ra và lưu trữ một cách an toàn trên thiết bị mà không bao giờ để lộ ra ngoài.
- **Chế độ bảo mật**: Các khóa này không thể truy cập trực tiếp từ phần mềm, chỉ có thể sử dụng thông qua các API được cung cấp. Điều này giúp tránh việc khóa bị rò rỉ hay đánh cắp.
- **Bảo vệ phần cứng**: Trên các thiết bị hỗ trợ **Secure Hardware** (phần cứng bảo mật, như Trusted Execution Environment - TEE hoặc Secure Element - SE), Keystore sử dụng các cơ chế phần cứng để bảo vệ các khóa. Trên các thiết bị không hỗ trợ phần cứng bảo mật, Keystore sử dụng phần mềm để bảo vệ khóa.

c. Tác dụng:

Việc chuyển từ thuật toán Caesar sang **AES-GCM-256** mang lại những cải thiện rõ rệt về bảo mật và độ tin cậy:

- Bảo mật nâng cao

+ **Mã hóa mạnh mẽ:** AES-GCM-256 là thuật toán mã hóa hiện đại, được chuẩn hóa và sử dụng rộng rãi, với chế độ GCM cung cấp cả tính bảo mật và xác thực dữ liệu.

+ **Đảm bảo tính toàn vẹn:** Chế độ GCM không chỉ mã hóa dữ liệu mà còn đảm bảo tính toàn vẹn của dữ liệu, giúp phát hiện mọi sửa đổi trái phép.

- Quản lý khóa an toàn với Android Keystore

+ **Tách biệt khóa mã hóa:** Keystore đảm bảo rằng khóa AES được lưu trữ và quản lý an toàn, giảm thiểu nguy cơ lộ lọt khóa do ứng dụng hoặc hệ điều hành bị xâm phạm.

+ **Cách ly bảo mật:** Trên các thiết bị hỗ trợ phần cứng bảo mật, các khóa được lưu trữ và xử lý trong môi trường an toàn (Trusted Execution Environment hoặc Secure Element), đảm bảo an ninh cao hơn.

+ **Chống trộm khóa:** Với cơ chế không để lộ khóa ra ngoài, Keystore bảo vệ chống lại việc đánh cắp khóa qua các phương pháp tấn công phần mềm hoặc phần cứng.

- Tăng cường khả năng quản lý dữ liệu

+ **Tính duy nhất của khóa:** Sử dụng **username** để tạo khóa AES độc lập cho mỗi người dùng, đảm bảo dữ liệu được mã hóa riêng biệt và không thể truy cập trái phép bởi các người dùng khác.

+ **Khả năng mở rộng:** Với phương thức `createUserKey`, hệ thống dễ dàng thêm người dùng mới mà không ảnh hưởng đến các khóa hiện có.

+ **Linh hoạt và bảo mật cao:** Tạo IV ngẫu nhiên cho mỗi lần mã hóa giúp đảm bảo rằng dữ liệu mã hóa là duy nhất ngay cả khi nội dung gốc giống nhau.

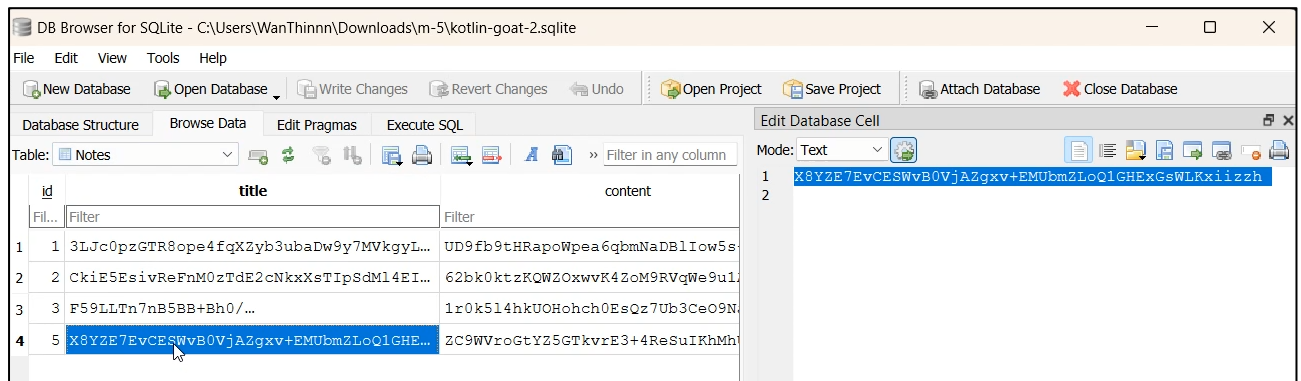
- Cải thiện trải nghiệm người dùng

+ **Giải mã tự động:** Dữ liệu được tự động giải mã khi người dùng đăng nhập, mang lại trải nghiệm liền mạch mà không cần tương tác thủ công.

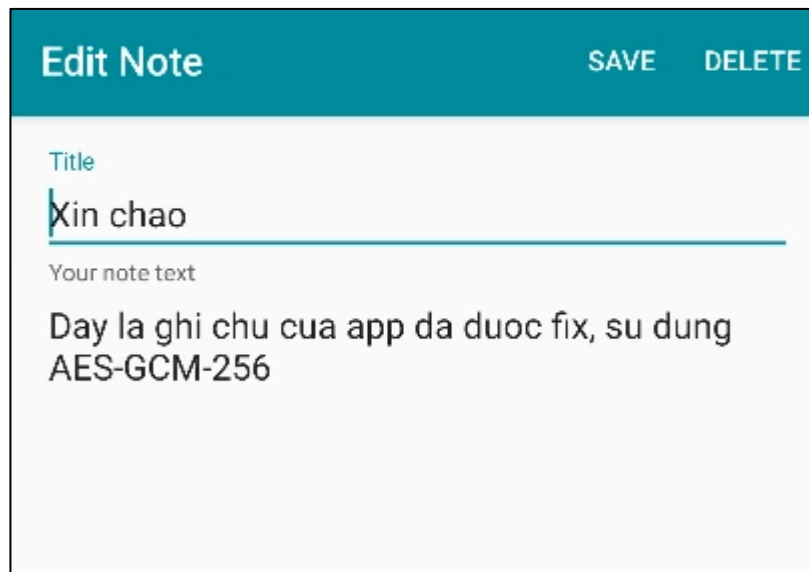
+ **Tăng cường bảo mật thông tin cá nhân:** Người dùng cảm thấy an tâm hơn khi biết dữ liệu của họ được bảo vệ tốt với công nghệ mã hóa tiên tiến.

d. Kết quả

Dữ liệu đã được lưu và mã hoá thành công:



Khi user đăng nhập vào, dữ liệu sẽ được tự động giải mã theo đúng với key của username đó:



3.2.8. M6: Insecure Authorization (Ủy quyền không an toàn)

a. Phân tích:

Mô tả lỗ hổng: Trong ứng dụng **Goatlin**, lỗ hổng **Insecure Authorization** xuất hiện do thiếu kiểm tra quyền hạn (authorization) trong các API endpoint, mặc dù đã có bước xác thực (authentication). Cụ thể:

- Authorization không được áp dụng khi người dùng thực hiện các yêu cầu đến API.

- Đây là lỗ hổng phổ biến khi hệ thống phụ thuộc vào IDOR (Insecure Direct Object Reference) hoặc khi giả định rằng endpoint chỉ được truy cập bởi người dùng có quyền hạn đúng.

```
router.put('/accounts/:username/notes/:note', auth, async (req, res, next) => {
```

```
router.get('/accounts/:username/notes', auth, async (req, res, next) => {
```

- Các endpoint trên chỉ kiểm tra xác thực (auth) mà không xác minh quyền sở hữu tài nguyên.

Mức độ ảnh hưởng của lỗ hổng: Bất kỳ người dùng nào sau khi đăng nhập có thể truy cập và quản lý tài nguyên không thuộc quyền sở hữu của mình.

b. Khắc phục:

Để khắc phục vấn đề này, ta bổ sung middleware **ownership** để kiểm tra quyền sở hữu tài nguyên. Middleware này thực hiện kiểm tra như sau:

```
function ownership (req, res, next) {  
  if (req.params.username !== req.account.username) {  
    res.statusMessage = "Unauthorized"  
    return res.status(403).end();  
  }  
  
  next();  
}  
  
module.exports = ownership
```

- Xác định người dùng hiện tại từ token xác thực.

- So sánh thông tin từ yêu cầu API (req.params.username) với thông tin của người dùng đã xác thực (req.account.username).

- Nếu không khớp, yêu cầu bị từ chối với mã lỗi **403 Unauthorized**.

Sau khi bổ sung kiểm tra quyền sở hữu, các endpoint API được cập nhật như sau:

```
router.put('/accounts/:username/notes/:note', [auth, ownership], async (req, res, next) => {  
  
  router.get('/accounts/:username/notes', [auth, ownership], async (req, res, next) => {
```

c. Tác dụng:

Cải tiến bảo mật:

- **Kết hợp authentication và authorization:** Không chỉ xác thực người dùng mà còn đảm bảo rằng người dùng có quyền hạn cần thiết cho hành động cụ thể.

- **Giảm rủi ro IDOR:** Người dùng không thể truy cập hoặc sửa đổi tài nguyên không thuộc quyền sở hữu của mình.

- **Bảo vệ API endpoint:** Quyền truy cập API được kiểm soát chặt chẽ, giảm nguy cơ bị khai thác bởi các tác nhân độc hại thông qua việc dò tìm endpoint.

Lợi ích:

- **Ngăn chặn truy cập trái phép:** Chỉ người dùng sở hữu tài nguyên mới được phép thực hiện các thao tác CRUD.

- **Tuân thủ nguyên tắc bảo mật tối thiểu:** Người dùng chỉ có thể thực hiện các hành động trong phạm vi quyền hạn của mình.

- **Tăng cường khả năng mở rộng:** Middleware kiểm tra quyền hạn có thể dễ dàng áp dụng cho các endpoint khác, đảm bảo tính nhất quán.

d. Kết quả: đã nêu ở trên.

3.2.9. M7: Client Code Quality (Chất lượng mã phía client kém)

a. Phân tích:

Mô tả lỗ hổng: Lỗ hổng thuộc nhóm Client Code Quality thường liên quan đến các vấn đề cấp mã nguồn như **Buffer Overflow** hoặc **DOM-based XSS** trong các ứng dụng di động sử dụng WebView. Nguyên nhân thường đến từ việc sử dụng sai API hoặc cấu trúc ngôn ngữ lập trình. Những lỗ hổng này yêu cầu kiến thức sâu về mã nguồn và cách khai thác ở mức thấp để thực hiện tấn công.

Mức độ ảnh hưởng của lỗ hổng: thực thi mã độc trong không gian địa chỉ của ứng dụng, các vấn đề như **Buffer Overflow** hoặc **Memory Leak** có thể dẫn đến tấn công từ chối dịch vụ (DoS) hoặc truy cập trái phép.

Để phát hiện lỗ hổng này, sử dụng các công cụ phân tích tĩnh như **Detekt** – một công cụ kiểm tra mã nguồn Kotlin. Detekt cung cấp báo cáo chi tiết về phức tạp hóa mã nguồn, lỗi coding style, và các "code smells".

Kết quả chạy Detekt trên mã nguồn Goatlin:

```
Overall debt: 7h

Complexity Report:
- 1230 lines of code (loc)
- 912 source lines of code (sloc)
- 610 logical lines of code (lloc)
- 87 comment lines of code (cloc)
- 145 McCabe complexity (mcc)
- 47 number of total code smells
- 9 % comment source ratio
- 237 mcc per 1000 lloc
- 77 code smells per 1000 lloc

Project Statistics:
- number of properties: 149
- number of functions: 75
- number of classes: 18
- number of packages: 5
- number of kt files: 17
```

```

Ruleset: complexity - 40min debt
    TooManyFunctions - 15/11 - [DatabaseHelper] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/helper
    ComplexMethod - 15/10 - [showProgress] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/LoginActivi
Ruleset: exceptions - 1h 40min debt
    TooGenericExceptionCaught - [exception] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/helpers/Da
    TooGenericExceptionThrown - [installDatabaseFromAssets] at goatlin/packages/clients/android/app/src/main/java/com/cx/go
    TooGenericExceptionThrown - [getAccount] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/helpers/D
    TooGenericExceptionThrown - [getNote] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/helpers/Data
    TooGenericExceptionCaught - [e] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/EditNoteActivity.k
Ruleset: style - 4h 10min debt
    MagicNumber - [lowerBoundary] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/helpers/CryptoHelper
    WildcardImport - [LoginActivity.kt] at goatlin/packages/clients/android/app/src/main/java/com/cx/goatlin/LoginActivity.

```

- Tổng quan:

+ 1230 dòng mã (LOC)

+ 47 lỗi "code smells"

+ Mức độ phức tạp McCabe: 237 trên mỗi 1000 dòng logic

- Chi tiết lỗi:

+ Complexity Ruleset:

- **TooManyFunctions:** DatabaseHelper có 15 hàm, vượt ngưỡng 11.
- **ComplexMethod:** Hàm showProgress phức tạp hơn mức cho phép.

+ Exceptions Ruleset:

- Quản lý ngoại lệ chưa cụ thể, ví dụ: **TooGenericExceptionThrown** trong nhiều phương thức của DatabaseHelper.

+ Style Ruleset:

- Sử dụng "MagicNumber" trong CryptoHelper.
- Import kí tự đại diện trong LoginActivity.

b. Khắc phục:

Giảm phức tạp mã nguồn:

- Tách DatabaseHelper thành các lớp nhỏ hơn, mỗi lớp chỉ đảm nhận một chức năng cụ thể.

- Tối ưu hóa các hàm phức tạp như showProgress bằng cách chia thành các hàm con.

Cải thiện quản lý ngoại lệ:

- Tránh sử dụng các ngoại lệ chung chung như Exception.

- Sử dụng các loại ngoại lệ cụ thể như SQLException hoặc IOException để giúp xác định và xử lý lỗi rõ ràng hơn.

Tuân thủ coding style:

- Loại bỏ "MagicNumber" bằng cách sử dụng các hằng số được định nghĩa trước.
- Hạn chế sử dụng import ký tự đại diện (*) để cải thiện tính rõ ràng và bảo trì của mã nguồn.

c. Tác dụng:

- **Tăng cường bảo mật:**
 - + Ngăn chặn khả năng khai thác lỗ hổng qua các lỗi như Buffer Overflow và Memory Leak.
 - + Bảo vệ dữ liệu người dùng và ngăn chặn thực thi mã độc.
- **Cải thiện chất lượng mã:**
 - + Dễ dàng bảo trì và mở rộng ứng dụng.
 - + Đảm bảo tuân thủ các tiêu chuẩn mã hóa trong tổ chức.
- **Tăng hiệu quả phát triển:**
 - + Giảm thời gian cần thiết để sửa lỗi và kiểm tra lại mã.
 - + Đảm bảo mã nguồn dễ đọc và dễ hiểu đối với các nhà phát triển mới.

d. Kết quả:

Sau khi thực hiện các cải tiến:

- Phức tạp hóa mã giảm rõ rệt, các hàm phức tạp đã được tách thành các thành phần nhỏ hơn.
- Quản lý ngoại lệ cụ thể hơn, giảm thiểu nguy cơ bỏ sót lỗi quan trọng.
- Tuân thủ quy tắc coding style, giảm thiểu lỗi "code smells".

3.2.10. M8: Code Tampering (Sửa đổi mã)

a. Phân tích:

Mô tả lỗ hổng: Code Tampering (Sửa đổi mã) là một điểm yếu bảo mật mà kẻ tấn công có thể khai thác để thay đổi mã của ứng dụng, cho phép truy cập trái phép hoặc thao túng chức năng của ứng dụng, khi Code Tampering xảy ra thì kẻ tấn công sẽ có các khả năng thực hiện thay đổi trực tiếp mã nhị phân, thay đổi tài nguyên cục bộ hoặc chuyển hướng hay thay thế API hệ thống

Mức độ ảnh hưởng của lỗ hổng: nghiêm trọng vì khi xuất hiện lỗ hổng này, ứng dụng có thể bị tác động kỹ thuật hoặc tác động về mặt uy tín hay kinh tế

b. Khắc phục:

Có nhiều phương pháp để khắc phục được lỗ hổng này như: kiểm tra tính toàn vẹn của mã, kiểm tra và giám sát runtime, sử dụng các công cụ bảo mật... Mỗi phương pháp có điểm mạnh riêng như:

- Kiểm tra tính toàn vẹn của mã:

- Phát hiện sớm việc mã bị sửa đổi: Sử dụng hàm băm (hash) như SHA-256 để kiểm tra tính toàn vẹn giúp xác định nhanh chóng nếu mã bị thay đổi
- Ngăn chặn mã độc: Phát hiện các thay đổi không mong muốn, ngăn mã bị chèn thêm mã độc
- Dễ triển khai: Các thư viện kiểm tra tính toàn vẹn phổ biến và dễ tích hợp vào quy trình phát triển phần mềm

- Kiểm tra và giám sát runtime Kiểm tra và giám sát runtime

- Phát hiện thời gian thực: Phát hiện hành vi bất thường khi ứng dụng đang chạy, chẳng hạn như việc tiêm mã (code injection) hoặc thay đổi mã
- Ngăn chặn khai thác: Ngăn chặn hành vi xâm nhập trước khi chúng gây thiệt hại lớn
- Tăng khả năng bảo mật: Cung cấp khả năng phát hiện và phản ứng nhanh trước các mối đe dọa tiềm năng

Trong các phương pháp kể trên thì nhóm sẽ sử dụng phương pháp tránh các thiết bị root/jailbreak sử dụng ứng dụng. Thông qua lớp **RootDetectionHelper** đã được nhóm phát triển và đề cập trước đó ở mục 3.2.2 thì nhóm sẽ thực hiện hàm **isDeviceRooted()** của lớp này ở ngay lớp ứng dụng được khởi tạo đầu tiên của ứng dụng (của nhóm là ở phần đăng nhập) để kiểm tra nếu thiết bị đã root/jailbreak thì sẽ ngăn chặn việc nó truy cập sử dụng ứng dụng

c. Tác dụng:

Việc sử dụng kiểm thử và ngăn chặn các thiết bị root/jailbreak mang lại nhiều lợi ích quan trọng cho ứng dụng như:

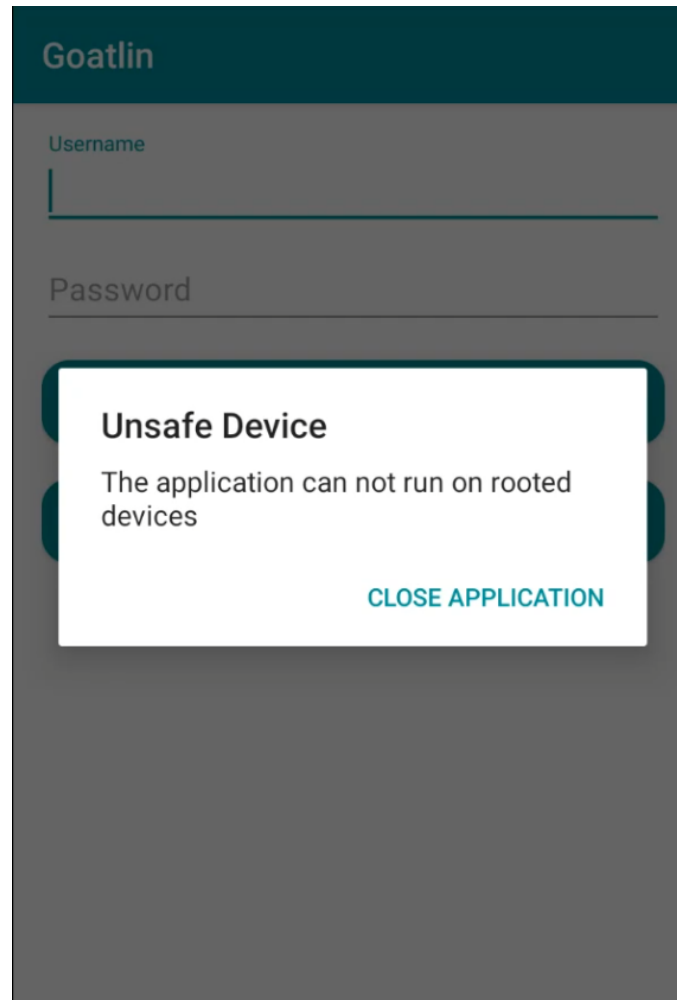
- **Bảo vệ dữ liệu và bảo mật:** Thiết bị đã root có thể dễ dàng bị tấn công bởi phần mềm độc hại hoặc các ứng dụng không an toàn. Bằng cách ngăn cản thiết bị root, ta giảm thiểu nguy cơ dữ liệu nhạy cảm bị đánh cắp hoặc bị lạm dụng

- **Đảm bảo tính toàn vẹn của ứng dụng:** Root có thể cho phép người dùng thay đổi hoặc can thiệp vào mã nguồn của ứng dụng, dẫn đến các hành vi không mong muốn hoặc không dự đoán trước. Ngăn cản thiết bị root giúp bảo vệ tính toàn vẹn và hoạt động ổn định của ứng dụng

- **Bảo vệ trải nghiệm người dùng:** Thiết bị root có thể gây ra các vấn đề về hiệu suất và ổn định. Bằng cách ngăn cản thiết bị root, ta đảm bảo rằng người dùng có trải nghiệm tốt nhất khi sử dụng ứng dụng

d. Kết quả:

Thiết bị đã bị block ngay khi ứng dụng kiểm tra khi mới bắt đầu mở ứng dụng lên



3.2.11. M9: Reverse Engineering (Kỹ thuật đảo ngược)

a. Phân tích:

Mô tả lỗ hổng: Reverse Engineering (Kỹ thuật đảo ngược) là quá trình phân tích cấu trúc, chức năng, hoặc hành vi của một ứng dụng để hiểu cách nó hoạt động, mà không có tài liệu hoặc mã nguồn chính thức từ nhà phát triển. Trong bảo mật ứng dụng di động, Reverse Engineering được sử dụng bởi kẻ tấn công để tìm kiếm điểm yếu trong ứng dụng, trích xuất thông tin nhạy cảm, hoặc thay đổi ứng dụng cho mục đích độc hại

Mức độ ảnh hưởng của lỗ hổng: rất nghiêm trọng vì khi Reverse Engineering xảy ra thì nó sẽ ảnh hưởng trực tiếp đến ứng dụng như tiết lộ thông tin nhạy cảm, đánh cắp tài sản trí tuệ hay thậm chí là tấn công vào hệ thống backend

b. Khắc phục:

Một số phương pháp dùng để khắc phục lỗ hổng này và có 1 số điểm mạnh riêng biệt như:

- Code Signing (Chứng thực mã)
 - Ngăn chặn giả mạo: Đảm bảo rằng ứng dụng không bị thay đổi sau khi phát hành
 - Tin cậy cao: Người dùng có thể kiểm tra tính hợp pháp của ứng dụng thông qua chứng thực
 - Yêu cầu tối thiểu từ phía nhà phát triển: Không cần thay đổi logic ứng dụng
- Binary Packing (Đóng gói nhị phân)
 - Bảo vệ mạnh mẽ: Khó giải nén và phân tích
 - Giảm khả năng sửa đổi mã: Làm tăng đáng kể thời gian cần để đảo ngược ứng dụng
 - Hiệu quả chi phí: Công cụ đóng gói thường có giá hợp lý và dễ sử dụng
- Secure APIs and Server-Side Logic
 - Bảo mật cao: Hacker không thể dễ dàng truy cập vào logic hoặc dữ liệu quan trọng

- Phù hợp cho ứng dụng nhạy cảm: Đặc biệt hiệu quả với ứng dụng yêu cầu xác thực hoặc xử lý thanh toán

Để khắc phục được lỗ hổng này thì nhóm sẽ thực hiện sử dụng Obfuscation (bật minifyEnabled) để làm rối mã từ đó làm cho mã nguồn khó hiểu và khó phân tích hơn khi ứng dụng bị dịch ngược

```

1  apply plugin: 'com.android.application'
2
3  apply plugin: 'kotlin-android'
4
5  apply plugin: 'kotlin-android-extensions'
6
7  android {
8      compileSdkVersion 26
9      defaultConfig { DefaultConfig it ->
10         applicationId "com.cx.vulnerablekotlinapp"
11         minSdkVersion 26
12         targetSdkVersion 26
13         versionCode 1
14         versionName "1.0"
15         testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"
16     }
17     buildTypes { NamedDomainObjectContainer<BuildType> it ->
18         release {
19             minifyEnabled true
20             proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
21         }
22     }
23     lintOptions { LintOptions it ->
24         abortOnError false
25     }
26 }
27

```

c. Tác dụng:

Khi thực hiện sử dụng Obfuscation thì ta đã gây được khó khăn cho những kẻ tấn công ít nhiều, từ đó giảm thiểu được các thiệt hại:

- **Bảo vệ tài sản trí tuệ:** Ngăn chặn kỹ thuật đảo ngược giúp bảo vệ mã nguồn và các thuật toán độc quyền, giữ vững lợi thế cạnh tranh của doanh nghiệp¹.

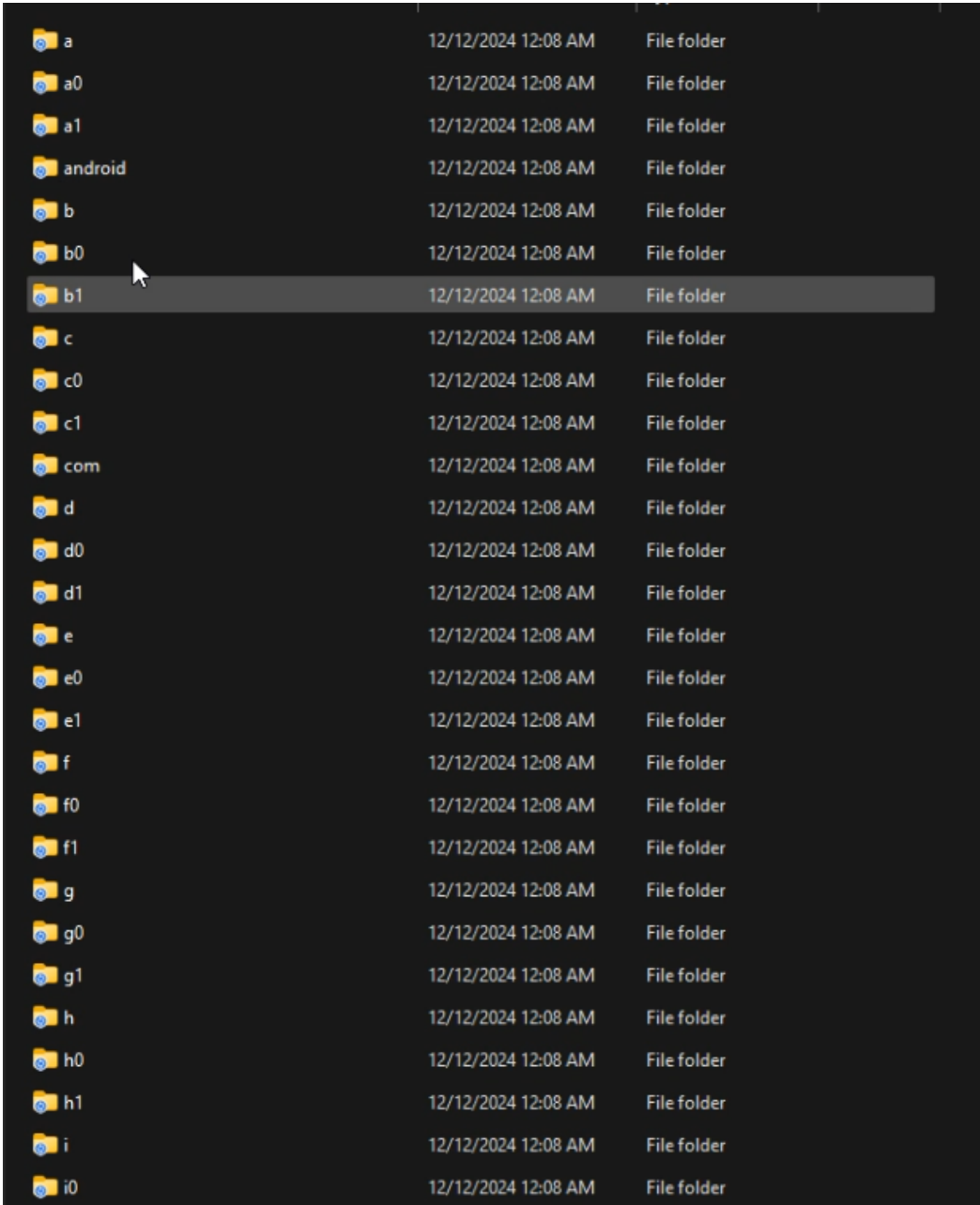
- **Bảo vệ thông tin nhạy cảm:** Đảm bảo rằng các thông tin nhạy cảm như khóa mã hóa và thông tin xác thực không bị lộ ra ngoài, giúp bảo vệ dữ liệu người dùng và hệ thống backend¹.

- **Ngăn chặn tấn công:** Giảm thiểu nguy cơ bị tấn công vào hệ thống backend và các dịch vụ liên quan, bảo vệ toàn vẹn và an toàn của ứng dụng.

- **Giảm thiểu rủi ro pháp lý:** Bảo vệ dữ liệu cá nhân và tuân thủ các quy định bảo mật giúp doanh nghiệp tránh được các vấn đề pháp lý và phạt tiền

d. Kết quả:

Có thể thấy khi thực hiện dịch ngược bằng apktool thì mã nguồn của chương trình chúng ta đã bị làm rối lên từ tên các lớp, đến tên các hàm trong mã nguồn



a	12/12/2024 12:08 AM	File folder
a0	12/12/2024 12:08 AM	File folder
a1	12/12/2024 12:08 AM	File folder
android	12/12/2024 12:08 AM	File folder
b	12/12/2024 12:08 AM	File folder
b0	12/12/2024 12:08 AM	File folder
b1	12/12/2024 12:08 AM	File folder
c	12/12/2024 12:08 AM	File folder
c0	12/12/2024 12:08 AM	File folder
c1	12/12/2024 12:08 AM	File folder
com	12/12/2024 12:08 AM	File folder
d	12/12/2024 12:08 AM	File folder
d0	12/12/2024 12:08 AM	File folder
d1	12/12/2024 12:08 AM	File folder
e	12/12/2024 12:08 AM	File folder
e0	12/12/2024 12:08 AM	File folder
e1	12/12/2024 12:08 AM	File folder
f	12/12/2024 12:08 AM	File folder
f0	12/12/2024 12:08 AM	File folder
f1	12/12/2024 12:08 AM	File folder
g	12/12/2024 12:08 AM	File folder
g0	12/12/2024 12:08 AM	File folder
g1	12/12/2024 12:08 AM	File folder
h	12/12/2024 12:08 AM	File folder
h0	12/12/2024 12:08 AM	File folder
h1	12/12/2024 12:08 AM	File folder
i	12/12/2024 12:08 AM	File folder
i0	12/12/2024 12:08 AM	File folder

 a\$a.smali	12/12/2024 12:08 AM	SMALI File	3 KB
 a.smali	12/12/2024 12:08 AM	SMALI File	5 KB
 b\$a.smali	12/12/2024 12:08 AM	SMALI File	4 KB
 b.smali	12/12/2024 12:08 AM	SMALI File	27 KB
 c.smali	12/12/2024 12:08 AM	SMALI File	3 KB
 d.smali	12/12/2024 12:08 AM	SMALI File	2 KB
 e.smali	12/12/2024 12:08 AM	SMALI File	3 KB
 f.smali	12/12/2024 12:08 AM	SMALI File	12 KB
 g.smali	12/12/2024 12:08 AM	SMALI File	13 KB
 h\$a.smali	12/12/2024 12:08 AM	SMALI File	3 KB
 h\$b.smali	12/12/2024 12:08 AM	SMALI File	9 KB
 h\$c.smali	12/12/2024 12:08 AM	SMALI File	7 KB
 h\$d.smali	12/12/2024 12:08 AM	SMALI File	9 KB
 h\$e.smali	12/12/2024 12:08 AM	SMALI File	7 KB
 h.smali	12/12/2024 12:08 AM	SMALI File	11 KB
 i\$a.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 i\$b.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 i\$c.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 i.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 j.smali	12/12/2024 12:08 AM	SMALI File	4 KB
 k.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 l.smali	12/12/2024 12:08 AM	SMALI File	3 KB
 m.smali	12/12/2024 12:08 AM	SMALI File	2 KB
 n.smali	12/12/2024 12:08 AM	SMALI File	1 KB
 o.smali	12/12/2024 12:08 AM	SMALI File	27 KB
 p.smali	12/12/2024 12:08 AM	SMALI File	13 KB

3.2.12. M10: Extraneous Functionality (Tính năng thừa hoặc không cần thiết)

a. Phân tích:

Mô tả lỗi hổng: Extraneous Functionality (Tính năng thừa) xảy ra khi ứng dụng di động chứa các chức năng không cần thiết hoặc không được dự định phát hành trong môi trường sản xuất. Những tính năng này có thể bao gồm mã thử nghiệm, các điểm cuối API không chính thức, hoặc các cấu hình dành cho môi trường thử nghiệm và phát triển

Mức độ ảnh hưởng của lỗi hổng: nghiêm trọng vì nó có thể cung cấp thông tin về hệ thống backend tạo điều kiện cho các hành động không được phép, cũng như ảnh hưởng đến việc truy cập trái phép vào các chức năng nhạy cảm

b. Khắc phục:

Ở trong mã nguồn của ứng dụng ban đầu thì bên trong sự kiện đăng nhập có xuất hiện một tài khoản được tạo sẵn cho người dùng, tức là không cần đăng kí thì ta vẫn có thể đăng nhập vào ứng dụng thông qua tài khoản có sẵn này

```
override fun doInBackground(vararg params: Void): Boolean? {  
    if ((mUsername == "Supervisor") and (mPassword == "MySuperSecretPassword123!")){  
        return true  
    }  
}
```

Thông thường loại lỗi hổng này sẽ khó có thể phát hiện nên trước khi release ứng dụng người phát triển nên:

- Loại bỏ các chức năng không cần thiết trước khi phát hành
 - Giảm nguy cơ khai thác: Loại bỏ chức năng thừa giúp hạn chế bề mặt tấn công
 - Dễ thực hiện: Chỉ cần quy trình kiểm tra mã trước khi phát hành
 - Tối ưu hiệu năng: Giảm kích thước mã, tăng hiệu suất ứng dụng
- Sử dụng kiểm tra mã nguồn (Code Review)
 - Phát hiện sớm lỗi: Giúp phát hiện các đoạn mã debug hoặc chức năng ẩn trước khi triển khai

- Tăng cường chất lượng mã: Loại bỏ mã không cần thiết, cải thiện tính sạch sẽ của mã

- Đối với loại lỗ hổng này thì nhóm sẽ khắc phục bằng cách loại bỏ những tính năng thừa hoặc không cần thiết này đi, đồng thời cẩn thận trong quá trình phát triển và phát hành ứng dụng, không tạo các hàm không cần thiết quá nhiều, cũng như kiểm thử lại toàn bộ mã nguồn trước khi phát hành ứng dụng để không lộ các thông tin nhạy cảm

c. Tác dụng:

Ngăn chặn được lỗ hổng Extraneous Functionality sẽ mang lại nhiều lợi ích quan trọng cho ứng dụng như:

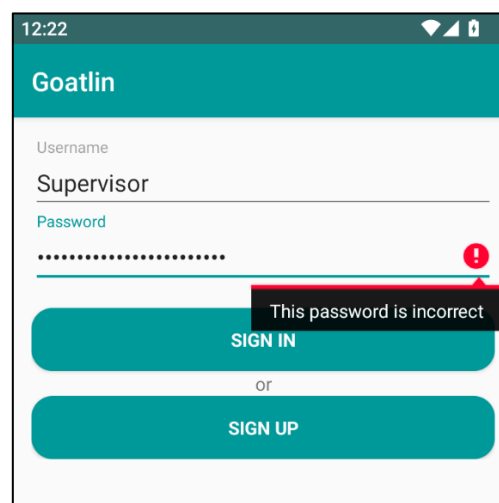
- **Bảo vệ thông tin hệ thống backend:** Ngăn chặn các chức năng thừa giúp bảo vệ thông tin về cách thức hoạt động của hệ thống backend, giảm nguy cơ bị tấn công.

- **Ngăn chặn hành động không được phép:** Loại bỏ các chức năng không cần thiết giúp ngăn chặn kẻ tấn công thực hiện các hành động có quyền cao mà không cần xác thực.

- **Bảo vệ dữ liệu nhạy cảm:** Đảm bảo rằng các chức năng thừa không tiết lộ dữ liệu nhạy cảm hoặc thông tin quan trọng, giúp bảo vệ dữ liệu người dùng và doanh nghiệp.

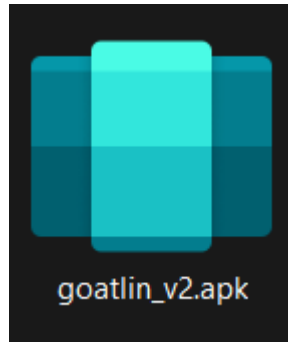
d. Kết quả:

Ta không thể sử dụng tài khoản có sẵn đó để đăng nhập vào ứng dụng được nữa



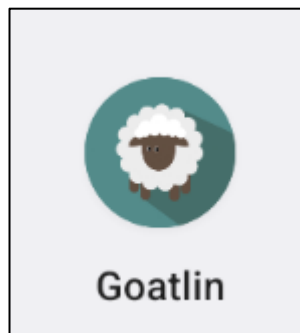
CHƯƠNG 4. TRIỂN KHAI VÀ KẾT QUẢ

Sau khi hoàn tất sửa đổi giao diện và tăng tính an toàn cho đoạn mã nguồn thì nhóm đã triển khai file apk cho ứng dụng an toàn thành công

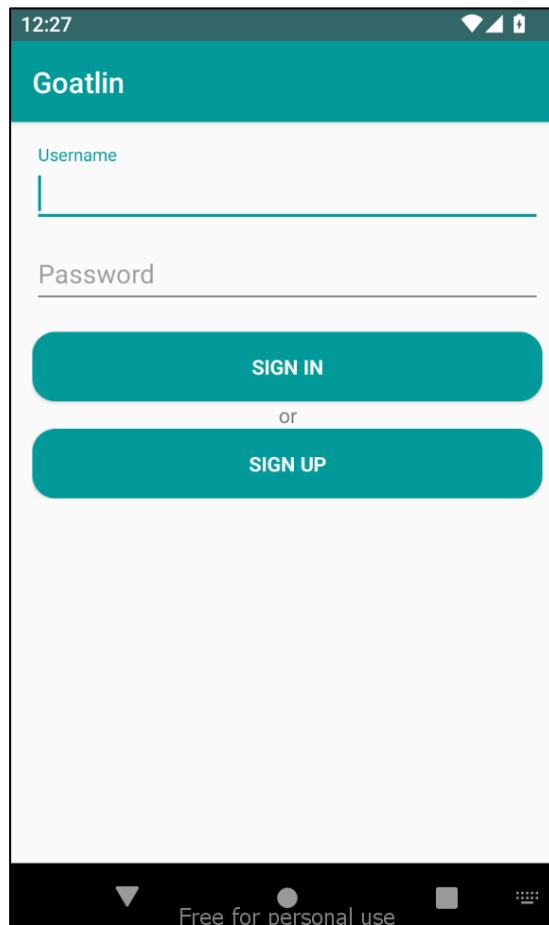


Luồng xử dụng của file sẽ tuân tự như sau:

- Sau khi thành công cài đặt file apk vào thiết bị, ứng dụng sẽ có tên là **Goatlin** và có ảnh nền là một chú cừu



- Vào app sẽ là giao diện đăng nhập, người dùng có thể thực hiện đăng nhập vào tài khoản nếu đã có sẵn



- Nếu chưa có tài khoản thì người dùng sẽ thực hiện tạo tài khoản bằng cách ấn vào nút Sign Up, lưu ý mỗi tài khoản mới được tạo phải thỏa mã 3 yêu cầu

- Tên người dùng chỉ được sử dụng chữ và chỉ có 1 khoảng cách 1 lần giữa những kí tự với nhau
- Tên tài khoản được tạo không được trùng với các tên tài khoản có sẵn
- Mật khẩu được tạo phải đảm bảo dài từ 10 đến 128 kí tự, có cả chữ in hoa và in thường, phải bao gồm thêm chữ số và có kí tự đặc biệt

Full name

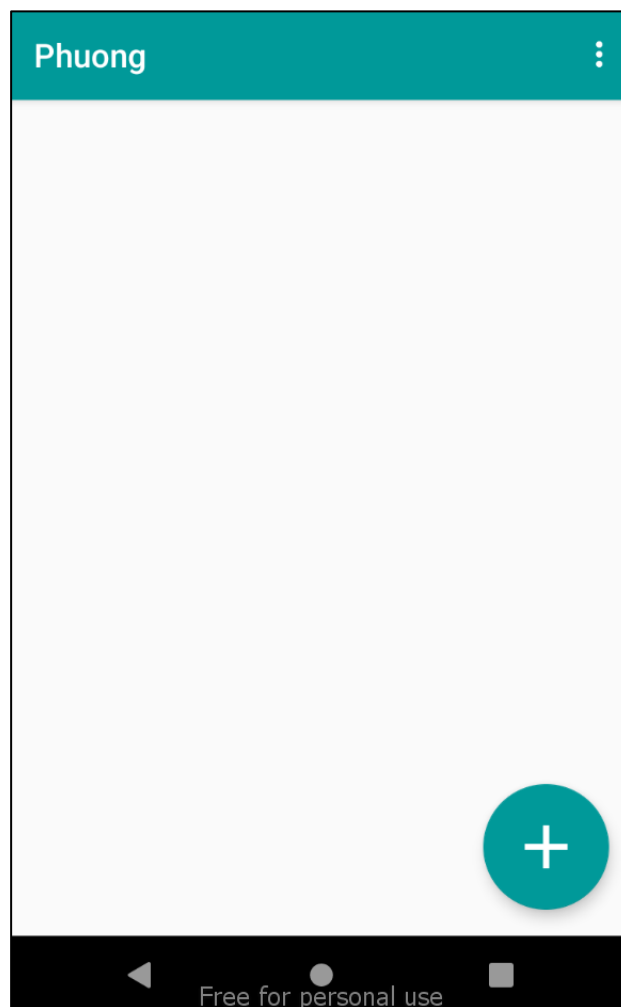
Username

Password

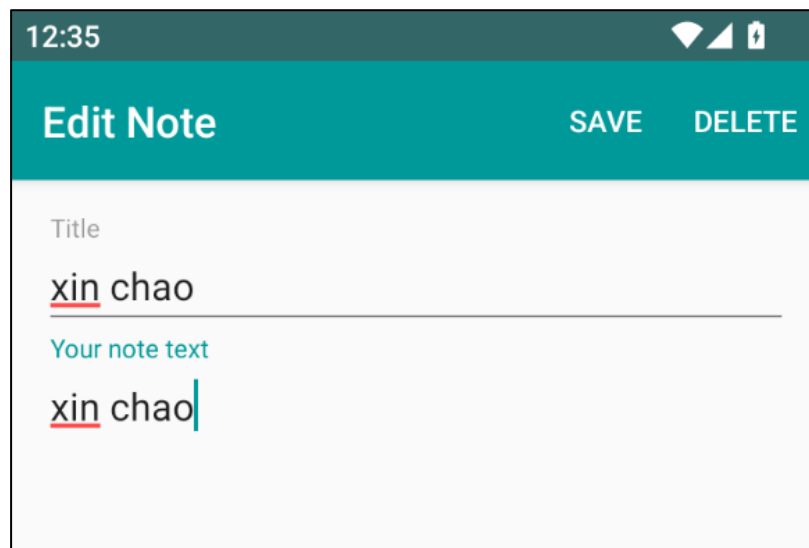
Confirm Password

SIGN UP

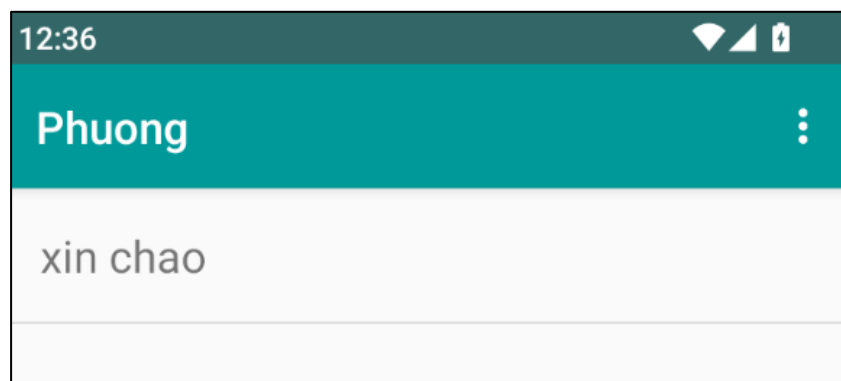
- Sau khi vào giao diện thì người dùng có thể tạo Note mới bằng nút có dấu cộng



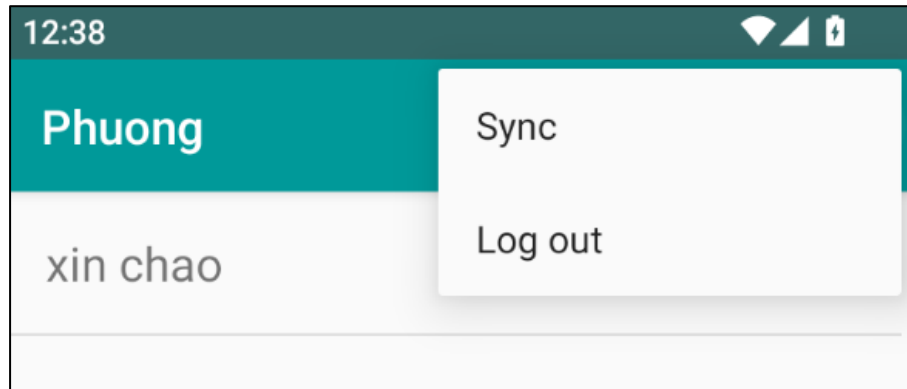
- Note sẽ không có điều kiện bắt buộc gì để đảm bảo toàn vẹn trải nghiệm riêng tư của người dùng, sau khi hoàn thành thì ấn Save để có thể lưu Note lại, Delete để xóa Note này



- Thông qua giao diện người dùng ban đầu thì những Note đã lưu sẽ được hiển thị ở đây, ta có thể click vào những Note ta muốn xem để có thể chỉnh sửa , hoặc xóa Note đó đi



- Ngoài ra ở nút 3 chấm của ứng dụng, ta còn cung cấp thêm cho người dùng 2 phương thức là đăng xuất (Log out), và lưu những Note mà ứng dụng có thể chưa cập nhật kịp (Sync)



CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Sau quá trình nghiên cứu và triển khai, ứng dụng **Goatlin** đã được phát triển thành công trên nền tảng Android, tích hợp các biện pháp bảo mật dựa trên tiêu chuẩn **OWASP Mobile Top 10** và các thuật toán mật mã tiên tiến như **AES-GCM**, **Bcrypt with SHA512**. Ứng dụng không chỉ đáp ứng các yêu cầu chức năng mà còn đảm bảo an toàn dữ liệu thông qua:

- Việc lưu trữ khóa bảo mật trong **Android Keystore**,
- Triển khai giao thức bảo mật truyền thông **HTTPS**,
- Bảo vệ dữ liệu khỏi các tấn công như **reverse engineering** và **code tampering**.

Kết quả này không chỉ minh chứng cho khả năng ứng dụng thực tế của các phương pháp bảo mật mà còn đóng góp vào việc nâng cao nhận thức và kỹ thuật trong việc xây dựng các ứng dụng di động an toàn.

5.2. Hạn chế

Trong quá trình thực hiện đồ án phát triển và bảo mật ứng dụng **Goatlin**, nhóm đã gặp phải một số khó khăn đáng kể, bao gồm:

- **Lần đầu tiếp xúc với mảng phát triển ứng dụng di động:** Do là lần đầu làm quen với lập trình ứng dụng trên Android, nhóm phải mất nhiều thời gian để tìm hiểu các công cụ như **Android Studio**, **Kotlin**, và cách thiết kế giao diện người dùng (UI/UX). Việc tích hợp bảo mật vào ứng dụng là một thách thức lớn khi phải đảm bảo cân bằng giữa tính năng và tính bảo mật.

- **Tối ưu hóa hiệu suất các thuật toán mật mã trên thiết bị di động:** Các thuật toán mật mã như **AES-GCM**, **RSA**, hay **SHA-256** vốn chạy rất nhanh trên desktop hoặc các thiết bị di động có hiệu năng cao. Khi áp dụng vào thiết bị di động với khả năng xử lý hạn chế, đặc biệt là các thiết bị đời cũ, việc đảm bảo tốc độ xử lý và tiết kiệm năng lượng trở thành một vấn đề khó khăn, đòi hỏi nhiều thời gian nghiên cứu và thử nghiệm.

- **Phân mảnh hệ sinh thái Android:** Hệ điều hành Android phân mảnh khá nhiều, với nhiều phiên bản khác nhau từ các nhà sản xuất như **ColorOS**, **HyperOS**, **OneUI**, v.v. Việc kiểm tra và tối ưu ứng dụng cho từng nền tảng và thiết bị đòi hỏi thời gian dài và tài nguyên phong phú, điều này vượt quá khả năng của nhóm trong khuôn khổ đề án.

- **Hạn chế về thời gian và tài nguyên:** Thời gian giới hạn cho đề án khiến nhóm không thể thực hiện đầy đủ các thử nghiệm và kiểm tra sâu rộng trên các thiết bị Android khác nhau. Các công cụ hỗ trợ kiểm thử và bảo mật chuyên sâu thường yêu cầu chi phí cao hoặc cần nhiều kinh nghiệm sử dụng.

Mặc dù có nhiều khó khăn, nhưng nhóm đã nỗ lực vượt qua để hoàn thành đề án, đồng thời rút ra được nhiều bài học kinh nghiệm quý giá trong việc phát triển và bảo mật ứng dụng di động.

5.3. Hướng phát triển trong tương lai

- **Cải thiện hiệu năng bảo mật:** Sử dụng **caching** hoặc các giải pháp tối ưu hóa mã nguồn.

- **Mở rộng phạm vi bảo mật:**

+ Áp dụng **biometrical authentication** (xác thực sinh trắc học) như vân tay hoặc nhận diện khuôn mặt.

+ Bảo vệ phiên người dùng bằng việc giám sát hoạt động bất thường hoặc đăng xuất tự động khi không sử dụng.

- **Khả năng đa nền tảng:** Phát triển ứng dụng trên các nền tảng khác như iOS hoặc xây dựng phiên bản web với các biện pháp bảo mật tương tự.

- **Nâng cấp giao diện người dùng:** Cải tiến trải nghiệm người dùng (UX) để dễ dàng tương tác hơn nữa, đồng thời nâng cao nhận thức về bảo mật thông qua hướng dẫn hoặc thông báo trong ứng dụng.

5.4. Lời kết

Đồ án phát triển và bảo mật ứng dụng **Goatlin** đã giúp nhóm nghiên cứu hiểu sâu hơn về cách xây dựng một ứng dụng di động an toàn, đáp ứng yêu cầu thực tiễn. Dự án không chỉ mang lại ý nghĩa trong học thuật mà còn mở ra hướng tiếp cận mới trong việc bảo mật ứng dụng di động.

Chúng em hy vọng rằng, với những cải tiến trong tương lai, Goatlin sẽ trở thành một ví dụ điển hình về ứng dụng an toàn, góp phần vào việc nâng cao ý thức bảo mật và bảo vệ người dùng trong thời đại công nghệ số.

----- **HẾT** -----

-